

Chapter 1: Introduction to Operating Systems

An operating system (OS) is the most fundamental software that runs on a computer. It manages hardware resources and provides a foundation for application programs. This chapter explains the core concepts, architecture, and services of modern operating systems, with real-life examples to ground the ideas.

What is an Operating System?

An operating system is a software layer that sits between the user (or application programs) and the computer hardware. Its main job is to abstract the messy details of hardware and provide a clean, convenient, and efficient environment for programs to execute.

Think of it as a hotel manager:

The guests (applications) ask for rooms, food, cleaning, etc. The manager (OS) decides who gets which room, when the kitchen operates, and ensures that guests don't disturb each other. The actual physical rooms, elevators, and kitchen (hardware) are managed by the staff without guests needing to know the plumbing or electrical wiring.

Examples of operating systems you encounter daily:

- **Windows** (desktops, laptops)
 - **Linux** (servers, Android phones, embedded devices)
 - **macOS** (Apple desktops/laptops)
 - **iOS** (iPhones, iPads)
 - **Android** (phones, tablets, smart TVs)
-

Roles of an Operating System

An OS plays three major roles.

1. Resource Manager

Hardware resources (CPU, memory, disk, network) are limited. The OS allocates them fairly and efficiently among competing programs.

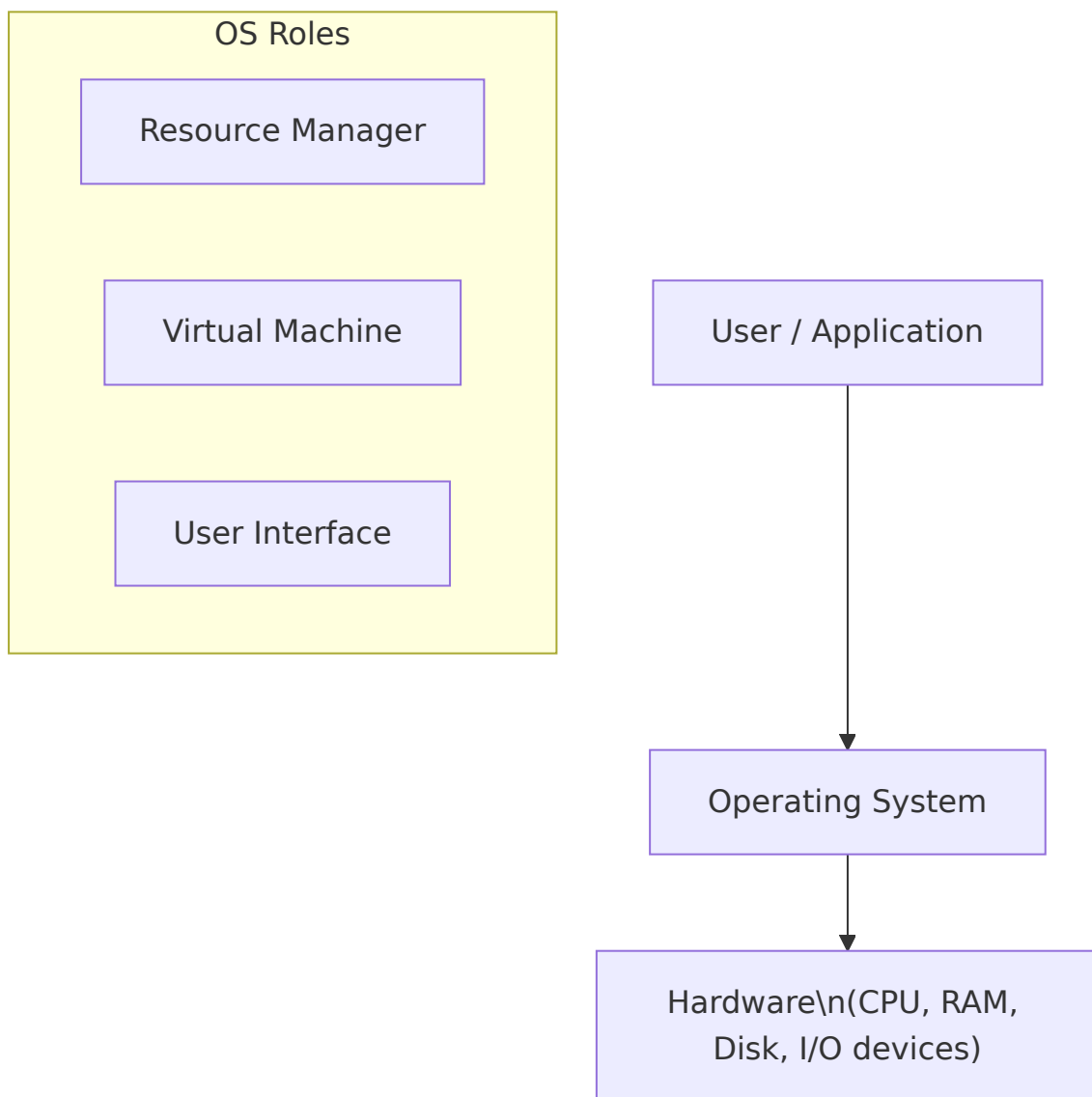
2. Virtual Machine

The OS creates an abstraction of the hardware that is simpler to program. For instance, you see files and folders instead of raw disk sectors; you see a window with a cursor instead of pixel buffers and interrupts.

3. Interface (between user and hardware)

The OS provides a user interface – command-line (CLI) or graphical (GUI) – through which you can run programs, manage files, and configure settings.

The following diagram shows the structural layers:



Computer System Architecture

A computer system consists of four main hardware components. They communicate over a system bus.

CPU (Central Processing Unit)

- Executes instructions (arithmetic, logic, control).
- Contains several cores (each can run a separate program/thread).
- Has a small, fast cache memory.

Memory (RAM – Random Access Memory)

- Stores data and instructions that the CPU actively uses.
- Volatile (loses content on power off).
- Divided into bytes, each with a unique address.

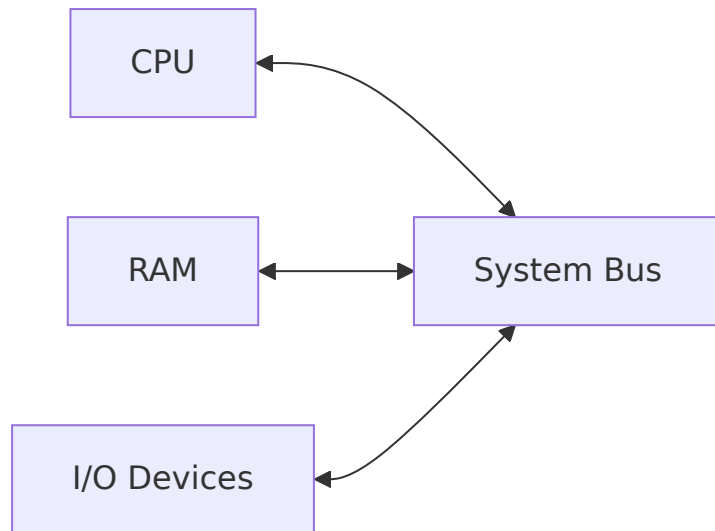
I/O Devices

- Input: keyboard, mouse, microphone, scanner.
- Output: monitor, printer, speakers.
- Storage: hard disk, SSD, USB drive (also treated as I/O).
- Network: Ethernet, Wi-Fi (specialised I/O).

System Bus

- The communication highway that connects CPU, memory, and I/O controllers.
- Includes address bus, data bus, and control bus.

Real-life analogy: A restaurant kitchen (CPU) prepares dishes using ingredients from a refrigerator (memory). Waiters (I/O) bring orders from customers and take finished food out. The kitchen pass (bus) is where orders and dishes are handed.



OS Structure: Monolithic, Layered, Microkernel, Hybrid

Operating systems are built in different architectural styles, each with trade-offs between performance, security, and maintainability.

Monolithic Kernel

All OS services run in a single kernel address space (kernel mode). This is fast because there is little overhead for communication, but a bug in any part can crash the whole system.

- **Examples:** Linux, older UNIX, MS-DOS.
- **Real-life:** A small shop where the owner does everything – cooks, cleans, handles money. Efficient but exhausting; one mistake spoils everything.

Layered Kernel

The OS is divided into layers, each built on top of the layer below. Lower layers provide services to upper layers. This improves modularity but can be slow due to layer-crossing overhead.

- **Example:** THE system (Dijkstra, 1968).

Microkernel

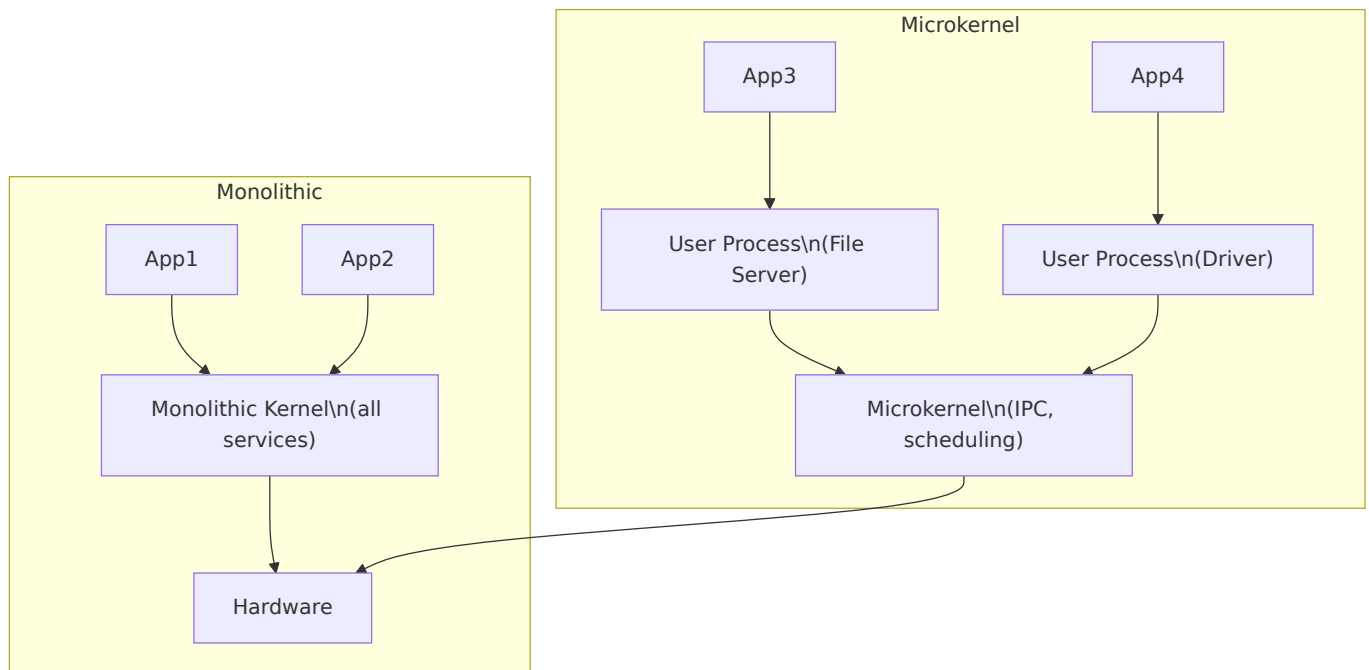
Only essential services (IPC, memory management, basic scheduling) run in kernel mode. Drivers, file systems, and networking run as user-mode processes. This is more secure and stable, but slower due to context switches.

- **Examples:** QNX, L4, MINIX (the inspiration for Linux' s early design).
- **Real-life:** A large organisation where each department (security, catering, IT) is independent. If catering fails, the company still runs.

Hybrid Kernel

A compromise – most services live in kernel for speed, but some run in user space. Combines the performance of monolithic kernels with some modularity of microkernels.

- **Examples:** Windows NT (and later Windows versions), macOS XNU.



User Mode vs Kernel Mode

CPUs have at least two privilege levels to protect the OS from buggy or malicious programs.

- **Kernel mode** (also called supervisor mode, ring 0):
 - Can execute **privileged instructions** (e.g., halt CPU, change memory mapping, access I/O devices directly).
 - Has access to the entire physical memory.
 - The OS kernel runs here.
- **User mode** (ring 3 on x86):
 - Cannot execute privileged instructions.
 - Access to memory is restricted to the process' s own address space.

- Applications run here.

How does the CPU know which mode it's in?

A **mode bit** in the CPU status register indicates kernel (0) or user (1).

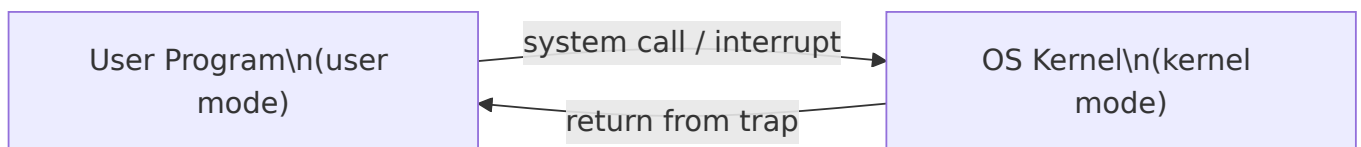
Switching from user mode to kernel mode:

- **System call** (e.g., read a file) – the program executes a special instruction (`syscall` , `int 0x80`).
- **Interrupt** (e.g., timer, keyboard press).
- **Exception** (e.g., division by zero, page fault).

When a switch happens, the CPU saves the current state and jumps to a predefined kernel handler. After handling, it returns to user mode.

Real-life analogy:

- **User mode:** Allowed to walk in the shopping mall (public areas). You can't enter the cash vault or the electrical room.
- **Kernel mode:** Security staff can go anywhere, open any door, and shut down the ventilation.
- **System call:** You ask a security guard to open a locked door for you.



System Calls and API

A **system call** is a request from a user program to the OS for a privileged service (e.g., open a file, create a process, allocate memory). System calls are the only way a user program can access kernel functionality.

Common system calls (Linux/UNIX):

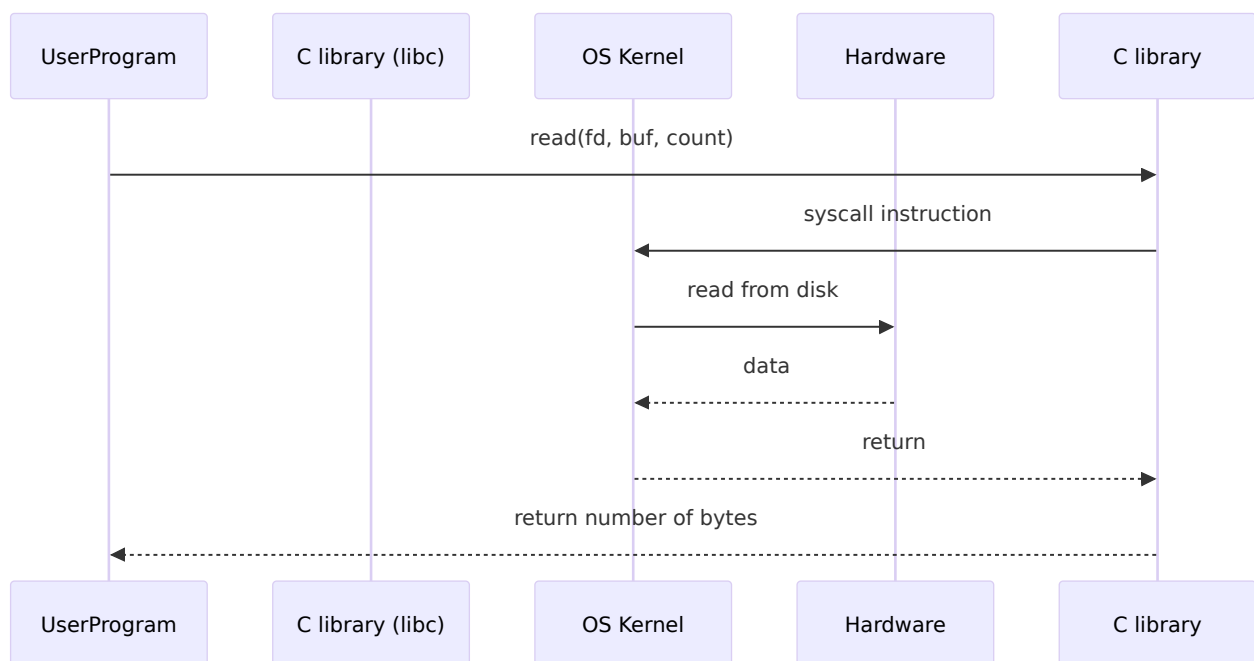
Category	Examples
Process control	<code>fork()</code> , <code>exec()</code> , <code>wait()</code> , <code>exit()</code>
File management	<code>open()</code> , <code>read()</code> , <code>write()</code> , <code>close()</code>

Device management	<code>ioctl()</code> , <code>read()</code> , <code>write()</code>
Information maintenance	<code>getpid()</code> , <code>time()</code>
Communication	<code>socket()</code> , <code>send()</code> , <code>recv()</code>

API (Application Programming Interface) – a set of functions provided by a library (like the C standard library) that often wraps system calls. An API may call one or more system calls, or do work without any system call.

- **Example:** `printf()` in C calls `write()` system call, but also does formatting.
- **Why use an API?** Portability – the same API can be implemented on different OSes with different system calls.

Sequence of a system call (e.g., reading from a file):



Real-life example:

You (application) want to withdraw money from a bank. You fill a withdrawal slip (API call). The teller (system call handler) checks your balance (kernel), goes to the vault (hardware), and gives you cash. You never enter the vault yourself.

OS Services and Utilities

Operating systems provide a set of core **services** that make life easier for users and programmers.

Service	Description	Example
Program execution	Load a program into memory and run it	Double-click an <code>.exe</code> or run <code>./a.out</code>
I/O operations	Read/write files, devices	<code>printf</code> , <code>scanf</code> , disk access
File system manipulation	Create, delete, rename files/directories	<code>mkdir</code> , <code>rm</code> , file explorer
Communication	Data exchange between processes (on same machine or network)	Pipes, sockets, shared memory
Error detection	Handle hardware errors, memory faults, division by zero	Segmentation fault handler
Resource allocation	Assign CPU time, memory, devices	Scheduler, memory manager
Accounting	Keep track of resource usage per user	<code>top</code> , <code>ps</code> , system logs
Protection	Prevent unauthorised access	File permissions, memory protection

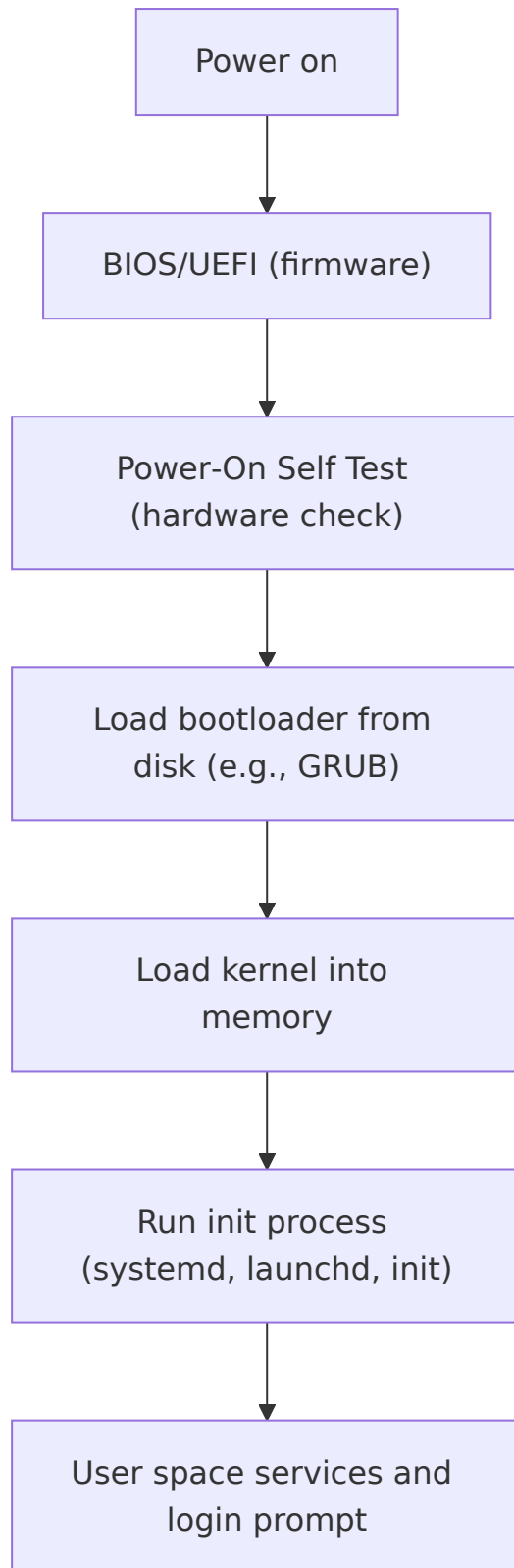
Utilities are user-level programs that come with the OS (e.g., `ls` , `cp` , `grep` , Task Manager). They use system calls to perform their work.

Real-life: A hotel provides services: luggage storage (resource allocation), wake-up calls (communication), laundry (I/O). Utilities are like the concierge desk – they rely on the hotel’ s internal systems.

Booting Process

Booting (short for “bootstrap”) is the sequence of operations that loads the operating system into memory and starts it when you turn on the computer.

Step-by-step boot flow



Detailed explanation

1. **Power-on** – The CPU starts in real mode (limited memory access) and jumps to a fixed memory address where the BIOS firmware resides.
2. **BIOS or UEFI** – Basic Input/Output System (older) or Unified Extensible Firmware Interface (modern) initialises the hardware: sets up clocks, initialises memory controller,

detects disks.

3. **POST (Power-On Self Test)** – Checks that critical hardware (CPU, RAM, disk controllers) works. If a fatal error occurs, it beeps or shows a message.
4. **Bootloader loading** – BIOS/UEFI reads the first sector of the boot disk (Master Boot Record, or EFI System Partition) and loads a small program called the bootloader (e.g., GRUB, Windows Boot Manager). The bootloader then loads a larger bootloader (stage 2) from the disk.
5. **Kernel loading** – The bootloader reads the OS kernel (e.g., `vmLinux` on Linux) into memory, possibly with an initial RAM disk (`initrd`) containing drivers needed to mount the real file system.
6. **Kernel initialisation** – The kernel starts, sets up CPU modes (switches to protected/long mode), initialises memory management, interrupt handlers, device drivers, and mounts the root file system (`/` on Linux).
7. **Init process** – The kernel starts the first user-mode process, traditionally `/sbin/init` (now often `systemd`, `launchd`, or `init`). This process brings up all other system services (networking, logging, graphical interface).
8. **Login** – Finally, the system presents a login prompt (text or graphical). After successful login, you get a shell or desktop environment.

Summary

Concept	Key takeaway
OS definition	Software that manages hardware, provides virtual machine, gives interface
Roles	Resource manager, virtual machine, interface
Computer architecture	CPU, RAM, I/O devices connected via bus
OS structures	Monolithic (fast, fragile), microkernel (stable, slower), hybrid (balanced)
User vs kernel mode	Privilege separation; system calls switch modes

System call	Request for kernel service; API abstracts it
OS services	Program execution, I/O, file system, communication, accounting, protection
Booting	Power-on → BIOS/UEFI → bootloader → kernel → init → login

Understanding these fundamentals is crucial before diving into process management, scheduling, and memory management – the topics of the next chapters.

Chapter 2: Process Management

A process is the core concept of any operating system. This chapter explains what a process is, how the OS manages it, and how modern systems use threads to improve performance and responsiveness.

Process Concept: Program vs Process

A **program** is a passive entity – a file on disk containing instructions and data (e.g., `a.out` , `chrome.exe`). A **process** is an active entity – a program in execution, with its own memory, registers, and state.

Key distinction:

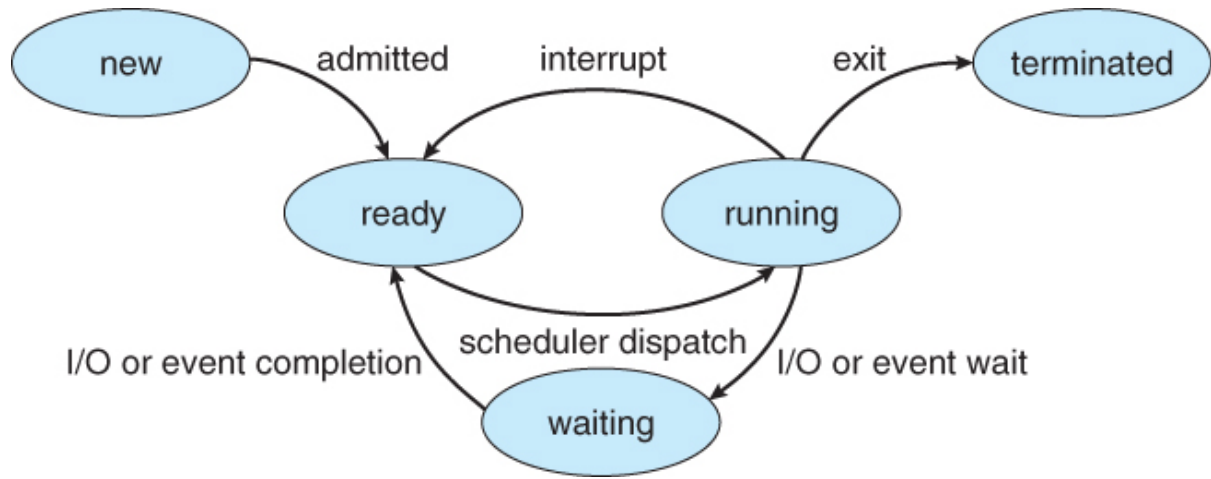
- Program: static, stored on disk, no resources allocated.
- Process: dynamic, loaded into memory, owned by the OS.

Real-life analogy: A recipe (program) is just text on paper. When you follow the recipe – heat the pan, chop onions, stir – that’ s a process. The same recipe can be cooked by many chefs simultaneously, each as a separate process.

Process States

As a process executes, it changes state. The OS tracks these states to manage CPU and memory efficiently.

State	Description
New	The process is being created (e.g., <code>fork()</code> called but not yet ready).
Ready	Process is in memory, waiting to be assigned to the CPU.
Running	Instructions are actually being executed on the CPU.
Waiting	Process is blocked, waiting for some event (I/O completion, signal).
Terminated	Process has finished execution (e.g., <code>exit()</code> called).



Real-life analogy: A customer in a restaurant:

- **New** – Enters the door, not yet seated.
- **Ready** – Waiting in queue for a table.
- **Running** – Sitting at table, eating.
- **Waiting** – Waiting for the waiter to bring the bill.
- **Terminated** – Leaves the restaurant.

Process Control Block (PCB)

For each process, the OS maintains a data structure called the **Process Control Block** (also called task control block). It contains all the information needed to manage the process.

Contents of PCB:

Field	Description
Process ID (PID)	Unique numerical identifier.
Program counter	Address of the next instruction to execute.
CPU registers	All general-purpose registers, stack pointer, etc.
Memory limits	Base and limit registers, page tables.
Open file list	File descriptors and their permissions.
I/O status	Devices assigned to this process.
Process state	New, ready, running, waiting, terminated.

Priority	Scheduling priority (if any).
Accounting info	CPU time used, process owner, etc.

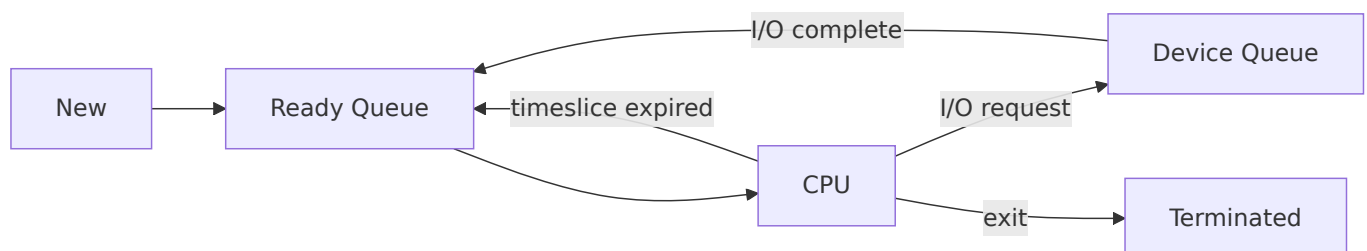
When the OS pauses a process (e.g., to run another), it saves the process' s CPU state into its PCB. When it resumes, it restores from the PCB.

Process Scheduling Queues

The OS maintains several queues to manage processes throughout their lifecycle.

Queue	Purpose
Job queue	All processes in the system (on disk or in memory).
Ready queue	Processes in the Ready state, waiting for CPU. Stored as a linked list of PCBs.
Device queue	Processes waiting for a specific I/O device (e.g., disk, printer). Each device has its own queue.

Flow: New process → Ready queue → CPU (Running) → if I/O → Device queue → back to Ready queue → Terminated.



Context Switching

Context switching is the act of saving the state (PCB) of the currently running process and loading the saved state of another process. It allows the CPU to switch between processes efficiently, giving the illusion of concurrency.

Steps:

1. Save CPU registers and program counter of the current process into its PCB.

2. Update the PCB state (e.g., from Running → Ready).
3. Move the PCB to the appropriate queue (ready or device queue).
4. Select another process from the ready queue (via scheduler).
5. Load its PCB into CPU registers, including program counter.
6. Start executing the new process.

Overhead: Context switching is pure overhead – the CPU does no useful work while switching. The cost depends on hardware support (how many registers to save) and memory management (TLB flush).

Real-life analogy: A professor grading assignments. When a student asks a question, the professor saves their place in the current paper (PCB), turns to the student (new process), answers, then returns to the previous paper (restores state).

Process Creation and Termination

Creation

A process can create another process using the `fork()` system call (Unix/Linux) or `CreateProcess()` (Windows). The creating process is the **parent**, the new one is the **child**.

`fork()` (Unix/Linux):

- Creates an exact copy of the parent process (text, data, heap, stack).
- Child gets a new PID and its own memory space.
- `fork()` returns:
 - `0` to the child process.
 - Child's PID to the parent.
 - `-1` on error.

`exec()` family – Replaces the current process' s memory with a new program. Typically used after `fork()` to run a different program.

```
pid_t pid = fork();
if (pid == 0) {
    // Child process
    execlp("/bin/ls", "ls", NULL);
} else {
    // Parent process
    wait(NULL);
}
```

`fork()` vs `exec()` :

- `fork()` clones the calling process.
- `exec()` loads a new program, discarding the old one.

Termination

A process terminates via `exit()` system call. Reasons:

- Normal completion (return from `main` or `exit`).
- Error or fatal condition.
- Killed by another process (e.g., `kill` on Unix).

When a process terminates, the OS:

- Releases all its resources (memory, open files, I/O buffers).
- Removes its PCB from queues.
- Informs the parent (via `wait()` or signal).

Zombie process: A child that has terminated but whose parent has not yet called `wait()`. The PCB is kept (to store exit status).

Orphan process: A child whose parent terminates before the child. On Unix, such children are adopted by the `init` process (PID 1), which calls `wait()` on them.

Process Hierarchies

Operating systems typically organise processes into **trees**. The root is the first process (e.g., `init` or `systemd` on Linux, `launchd` on macOS).

- **Parent** creates **children**.
- Children can create their own children – a hierarchy.
- The OS can propagate signals (e.g., `SIGTERM`) down the tree.

Example (Unix):

```
init (PID 1)
├─ systemd-logind
├─ sshd
│   └─ bash (user login)
│       └─ gcc (compiler)
└─ chrome
    ├─ chrome (tab process)
    └─ chrome (GPU process)
```

Real-life analogy: A company CEO (init) who hires managers (parents), who hire employees (children). If a manager leaves, employees may be reassigned to the CEO (orphan adoption).

Process vs Thread

A **thread** is a lightweight unit of execution within a process. A process has at least one thread (the main thread). Multiple threads within the same process share the process' s memory and resources.

Feature	Process	Thread
Memory space	Separate address space per process	Shared address space within a process
Resource overhead	High (PCB, page tables, file descriptors)	Low (only stack, registers, program counter)
Creation time	Slow (copy memory, set up data structures)	Fast (share most resources)
Context switch	Expensive (TLB flush, memory remapping)	Cheap (same address space, no TLB flush)
Communication	IPC (pipes, sockets, shared memory)	Direct via shared variables

Isolation	Strong (one process cannot corrupt another)	Weak (one thread can crash the whole process)
Examples	Running two browser windows (separate processes)	Tab inside a browser (multiple threads)

Real-life analogy:

- **Process** = An entire house: separate kitchen, bathroom, bedroom – isolated from other houses.
- **Thread** = A person living in that house: all people share the kitchen, bathroom, etc. They can talk directly, but one person burning the kitchen affects everyone in the house.

User-Level Threads vs Kernel-Level Threads

Threads can be implemented at two different levels.

User-Level Threads (ULT)

- Managed entirely by a user-space library (e.g., POSIX Pthreads on some implementations).
- The kernel is unaware of their existence – it sees only a single process.
- **Advantages:** Very fast creation and switching (no kernel mode switch). Portable.
- **Disadvantages:** If one thread blocks on I/O, the entire process blocks (the kernel cannot schedule another thread from the same process). Cannot utilise multiple CPU cores.

Kernel-Level Threads (KLT)

- Managed directly by the OS kernel. The kernel knows each thread individually.
- **Advantages:** True concurrency across cores; one thread blocking does not block others.
- **Disadvantages:** Slower to create and switch (requires system calls). More overhead per thread.

	User-Level	Kernel-Level
Management	User library	OS kernel
Switch speed	Very fast (no kernel mode)	Slower (needs syscall)

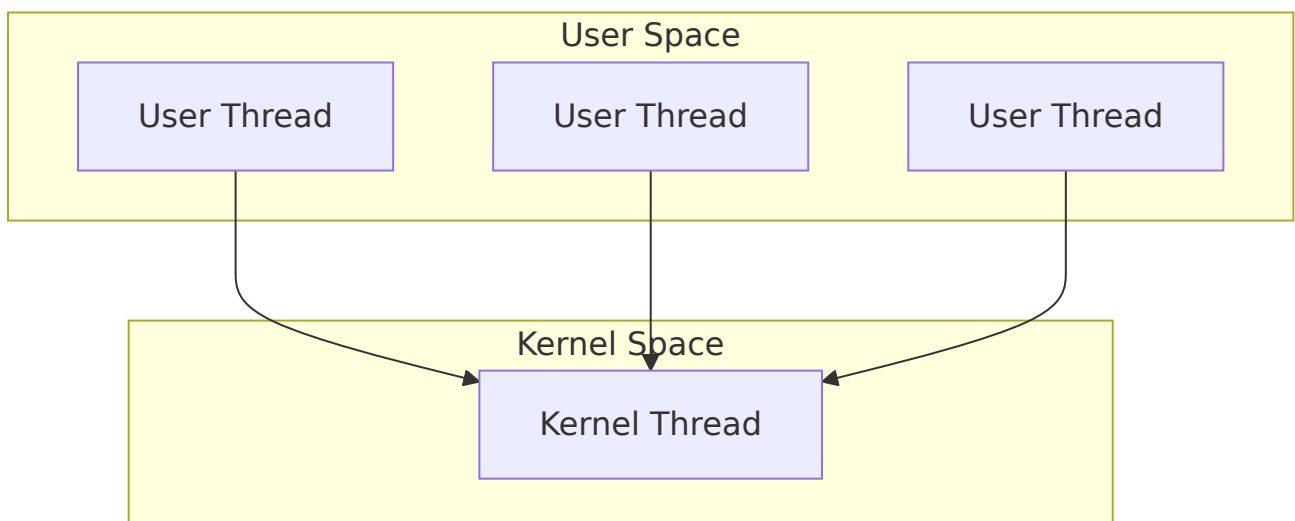
Blocking I/O	Blocks entire process	Blocks only that thread
Multiprocessor support	None (process runs on one CPU)	Full (threads on different cores)
Typical example	Old Java green threads	Modern Linux (NPTL), Windows threads

Multithreading Models

The relationship between user-level threads and kernel-level threads is defined by three classic models.

1. Many-to-One Model

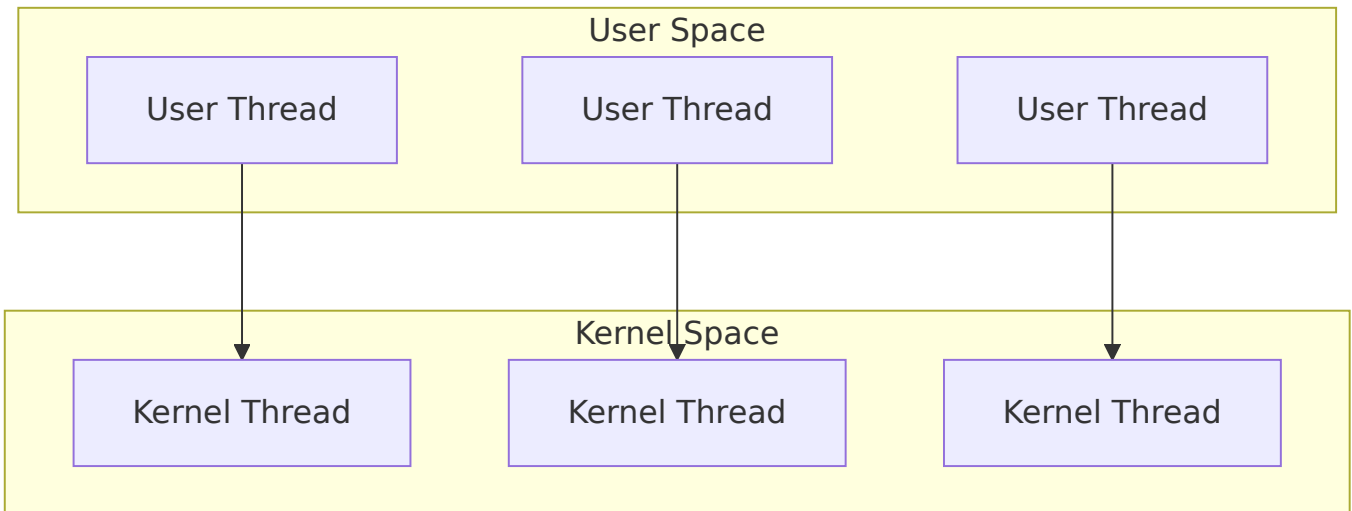
- Many user threads mapped to a single kernel thread.
- The kernel sees only one thread per process.
- **Pros:** Very lightweight.
- **Cons:** No parallelism (only one thread runs at a time); blocking I/O stops all.
- **Example:** Ancient Unix thread libraries (obsolete).



2. One-to-One Model

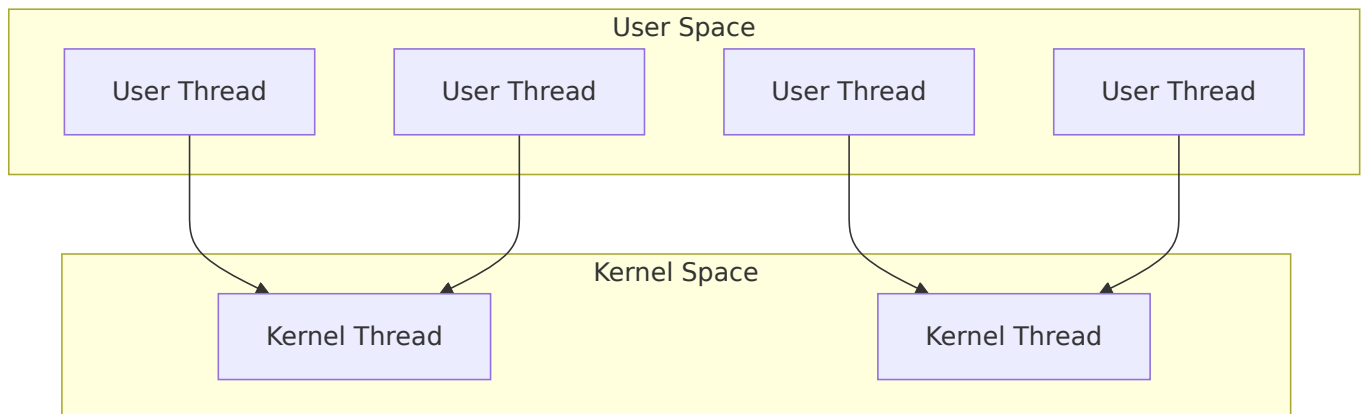
- Each user thread maps to a distinct kernel thread.
- **Pros:** True parallelism (each user thread can run on a different core); blocking I/O does not affect others.
- **Cons:** Creates kernel overhead per thread; limited by kernel thread limits.

- **Examples:** Linux (NPTL), Windows, macOS.



3. Many-to-Many Model

- Many user threads multiplexed onto a smaller or equal number of kernel threads.
- Allows parallelism up to the number of kernel threads.
- **Pros:** Balances performance and overhead; user threads can be created freely.
- **Cons:** Complex to implement (scheduler activations needed).
- **Example:** Original Solaris, older hybrid implementations (now largely replaced by one-to-one due to cheap kernel threads).



Two-level model (variation): Allows a user thread to be bound to a specific kernel thread (e.g., for real-time tasks), while others are multiplexed.

Summary

Concept	Key takeaway
---------	--------------

Program vs process	Program is passive on disk; process is active in memory with its own state.
Process states	new → ready → running → waiting → terminated.
PCB	Kernel data structure storing all per-process information.
Scheduling queues	Job, ready, device queues to manage process flow.
Context switching	Saving and restoring PCB to switch CPU between processes.
Process creation	<code>fork()</code> clones, <code>exec()</code> replaces memory image.
Process termination	<code>exit()</code> releases resources; parent <code>wait()</code> for status.
Process hierarchy	Tree structure rooted at <code>init</code> / <code>systemd</code> .
Process vs thread	Threads share memory within a process; threads are lighter.
User vs kernel threads	ULT: user-managed, fast, no parallelism; KLT: kernel-managed, slower, full concurrency.
Multithreading models	Many-to-one (obsolete), one-to-one (modern), many-to-many (compromise).

Understanding processes and threads is essential before tackling CPU scheduling and memory management – the next chapters in the journey.

Chapter 3: CPU Scheduling

CPU scheduling is the foundation of multitasking. The operating system decides which process (or thread) gets to use the CPU and for how long. This chapter explains the goals, algorithms, and trade-offs that make modern computers feel responsive and efficient.

Scheduling Criteria

To compare scheduling algorithms, we use several measurable criteria. The OS designer tries to optimise (maximise or minimise) these values.

Criterion	Definition	Goal
CPU utilisation	Percentage of time the CPU is busy executing useful work.	Maximise (keep CPU as busy as possible).
Throughput	Number of processes completed per unit time (e.g., jobs per second).	Maximise.
Turnaround time	Total time from process arrival to completion (waiting + execution + I/O).	Minimise.
Waiting time	Total time a process spends waiting in the ready queue.	Minimise.
Response time	Time from request submission (e.g., keystroke) to first response (not completion).	Minimise (important for interactive systems).

Real-life analogy: A coffee shop:

- **CPU utilisation** – How busy the barista is (80% busy is good, 100% means customers wait).
- **Throughput** – Number of coffees made per hour.
- **Turnaround time** – Time from when you enter the queue until you drink your coffee.
- **Waiting time** – Time you spend in the queue before the barista starts your order.
- **Response time** – Time from when you order until the barista says “I’ll start right away” (first acknowledgment).

Preemptive vs Non-Preemptive Scheduling

Non-preemptive scheduling: Once a process gets the CPU, it holds it until it either terminates or voluntarily yields (e.g., by waiting for I/O). The scheduler cannot forcibly remove it.

Preemptive scheduling: The scheduler can interrupt a running process and give the CPU to another. This decision can be based on a timer (time quantum) or a higher-priority process arriving.

Aspect	Non-preemptive	Preemptive
Control	Process decides when to release CPU	OS decides when to take CPU away
Overhead	Low (fewer context switches)	Higher (more context switches)
Fairness	Poor – a long process can starve others	Good – each process gets a slice
Response time	Poor for interactive tasks	Excellent
Example	Batch systems (old mainframes)	All modern OSes (Linux, Windows, macOS)

Real-life analogy:

- **Non-preemptive** – A meeting room: the first team that books it stays until they leave voluntarily.
- **Preemptive** – A classroom with a bell: every 50 minutes the class changes, regardless of whether the teacher wants to continue.

Note: Most modern schedulers are **preemptive** – they use timer interrupts to enforce fair CPU sharing.

Scheduling Algorithms

We now examine classic CPU scheduling algorithms. Each has strengths and weaknesses.

First-Come, First-Served (FCFS)

The simplest algorithm: processes are executed in the order they arrive – like a queue (FIFO).

- **Non-preemptive.**
- **Advantage:** Easy to implement.
- **Disadvantage:** The “convoy effect” – a long process blocks all shorter ones behind it, leading to high average waiting time.

Example (arrival times: all at time 0):

Process	Burst time
P1	24
P2	3
P3	3

Gantt chart:

| P1 | P2 | P3 |

Waiting times: P1 = 0, P2 = 24, P3 = 27. Average = $(0+24+27)/3 = 17$.

Real-life: A single checkout counter in a store – customers served in order. If one customer has a full trolley, everyone behind waits.

Shortest Job First (SJF)

Run the process with the **smallest next CPU burst** first. It is provably optimal for minimising average waiting time (for non-preemptive).

Non-Preemptive SJF

Once a process starts, it runs to completion. At each scheduling decision, the scheduler picks the shortest job among the arrived processes.

Example (arrival times: all at 0):

Process	Burst time
---------	------------

P1	6
P2	8
P3	7
P4	3

Order: P4 (3), P1 (6), P3 (7), P2 (8).

Waiting times: P4 = 0, P1 = 3, P3 = 9, P2 = 16. Average = 7 (much better than FCFS).

Preemptive SJF (Shortest Remaining Time First – SRTF)

If a new process arrives with a burst shorter than the **remaining time** of the current process, the scheduler preempts the current one. This yields even better average waiting time.

Example (with arrival times):

Process	Arrival	Burst
P1	0	8
P2	1	4
P3	2	9
P4	3	5

Gantt (SRTF):

| P1 | P2 | P4 | P1 | P3 |

Timeline: 0-1(P1), 1-5(P2), 5-10(P4), 10-17(P1), 17-26(P3).

Average waiting time = $(10-1) + (1-1) + (17-2) + (5-3)$... calculate – usually minimal.

Disadvantage: Starvation – long jobs may never run if short jobs keep arriving. Also, needing accurate future burst time estimation is difficult (but can be predicted using exponential averaging).

Real-life analogy: A hospital emergency room – triage treats the most critical (shortest) patients first, but a new critical arrival can preempt a less critical one.

Round Robin (RR)

Designed for time-sharing systems. Each process gets a small unit of CPU time called a **time quantum** (or timeslice, typically 10–100 ms). After the quantum expires, the process is preempted and moved to the back of the ready queue.

- **Preemptive** (by timer).
- **Advantage:** Good response time and fairness.
- **Disadvantage:** Performance heavily depends on quantum size.
 - Too large → degenerates to FCFS.
 - Too small → too many context switches, high overhead.

Rule of thumb: Quantum should be large compared to context-switch time (e.g., 10–100 ms vs ~1–10 μ s switch).

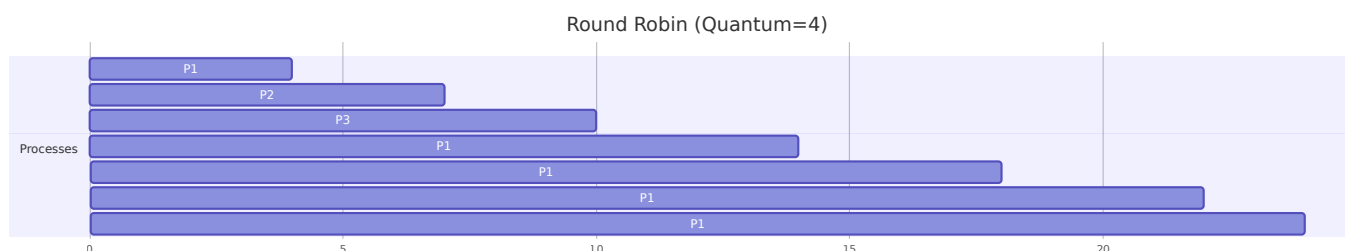
Example: Quantum = 4 ms. Processes arrive at time 0: P1 (burst 24), P2 (3), P3 (3).

Gantt:

| P1 | P2 | P3 | P1 | P1 | P1 | P1 | P1 |

(4)(4)(4)(4)(4)(4)(4) – Actually P2 and P3 finish quickly. Detailed: 0-4 P1, 4-7 P2, 7-10 P3, 10-14 P1, 14-18 P1, 18-22 P1, 22-24 P1.

Waiting times: P1 = (4+10-4? Better to compute formally – but concept clear.)



Priority Scheduling

Each process is assigned a priority (integer). The CPU is allocated to the process with the **highest priority** (assuming lower number = higher priority).

- Can be **preemptive** or **non-preemptive**.
- **Problem: Starvation** – low-priority processes may never run.
- **Solution: Aging** – gradually increase the priority of waiting processes over time.

Example (non-preemptive, higher number = higher priority):

Process	Burst	Priority
---------	-------	----------

P1	10	3 (low)
P2	1	1 (high)
P3	2	2 (medium)

Order: P2, P3, P1. Waiting times: P2=0, P3=1, P1=3. Average = 1.33.

With **aging**: If P1 waits too long, its priority rises (3→2→1) so it eventually gets CPU.

Real-life: Airport boarding – first class (high priority) boards before economy. But after some waiting, economy might be called (aging).

Multilevel Queue Scheduling

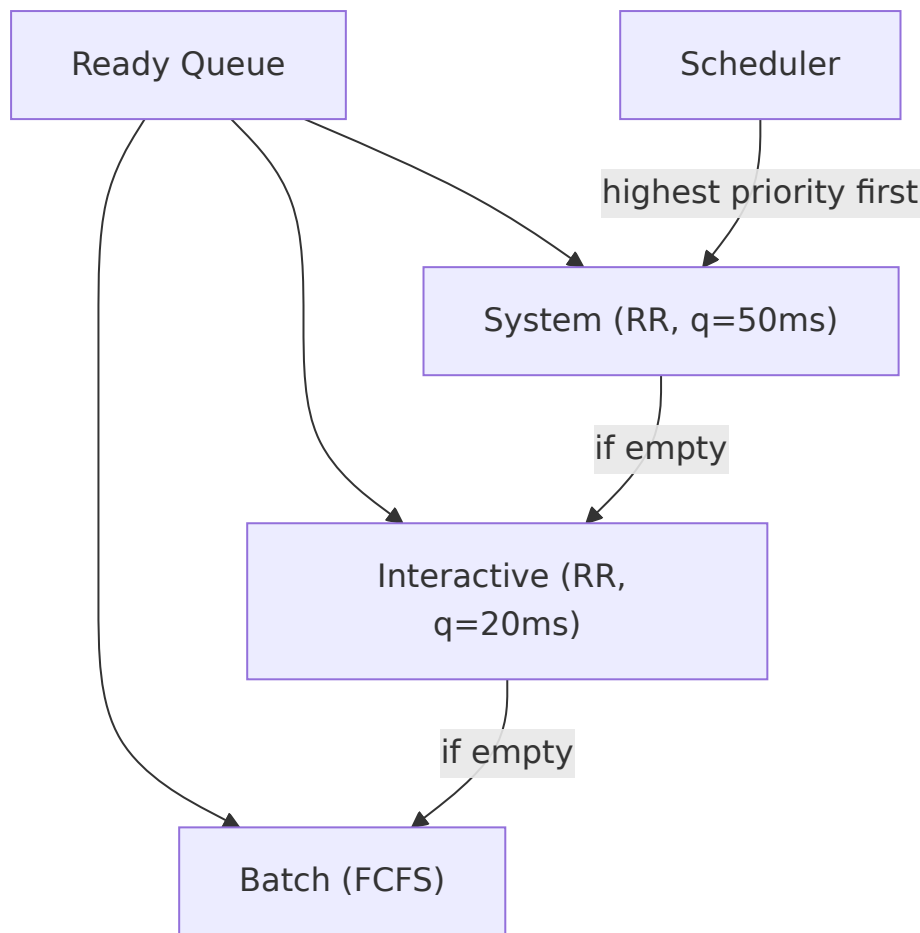
Processes are categorised into different **queues** based on their type (e.g., foreground interactive, background batch). Each queue has its own scheduling algorithm.

- **Permanent** assignment – a process stays in its queue.
- **Scheduling between queues**: Usually fixed-priority preemptive – higher-priority queues run first.

Example:

- Queue 0: **System processes** – RR, quantum 50 ms.
- Queue 1: **Interactive (foreground)** – RR, quantum 20 ms.
- Queue 2: **Batch (background)** – FCFS.

If Queue 0 is non-empty, it runs; if empty, Queue 1 runs; otherwise Queue 2.



- **Advantage:** Clear separation, suitable for systems with obvious process classes.
- **Disadvantage:** Inflexible – a process cannot change queues.

Multilevel Feedback Queue (MLFQ)

The most general and widely used scheduling algorithm (e.g., in classic UNIX, modern BSD, and Windows). It **dynamically** moves processes between queues based on their behaviour.

Rules (typical):

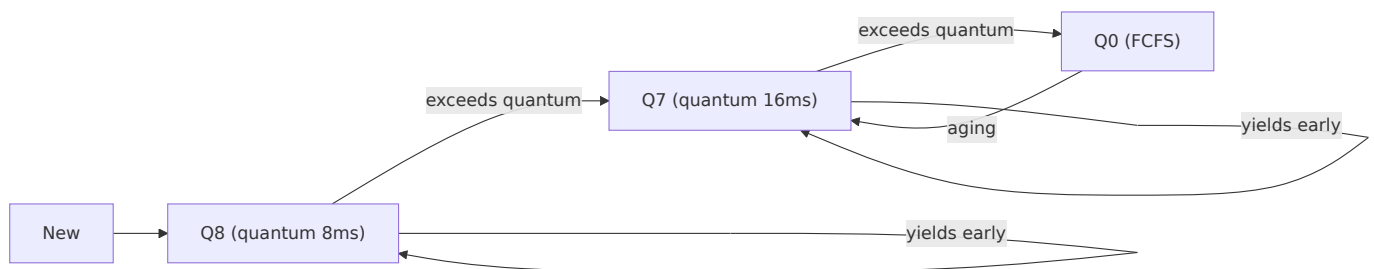
1. Multiple queues with different priorities. Higher-priority queues get shorter time quanta.
2. New processes enter the highest-priority queue.
3. If a process uses its entire quantum, it drops to a lower-priority queue.
4. If a process yields the CPU before quantum expires (e.g., for I/O), it stays in the same queue or moves up.
5. After a long waiting time, a process can be boosted to a higher queue (aging).

Example configuration:

- Q8 (highest priority): RR, quantum 8 ms – for interactive (short) jobs.
- Q7: RR, quantum 16 ms.
- ...
- Q0: FCFS – for CPU-bound long jobs.

Result:

- I/O-bound (interactive) processes get fast response.
- CPU-bound processes gradually fall to lower queues, not starving interactive ones.



Real-life: A restaurant with multiple lines – a quick coffee order goes to the express counter (high priority). If an order takes too long, it moves to a slower line. Orders that have been waiting too long get expedited.

Thread Scheduling

Modern operating systems schedule **threads**, not processes. The distinction depends on the threading model (see Chapter 2).

- **User-level threads (ULT):** The kernel schedules the **process** that contains many user threads. The user thread library internally schedules threads onto the single kernel thread. The kernel is unaware of the threads – so one blocking thread blocks the entire process.
- **Kernel-level threads (KLT):** The kernel schedules each thread individually. This allows true parallelism across cores and independent blocking.

Process-Contention Scope (PCS): Scheduling among user threads within the same process (managed by user library).

System-Contention Scope (SCS): Scheduling among kernel threads across all processes (managed by OS).

Example: In Linux (NPTL), `pthread_create` creates kernel threads. The scheduler uses the same algorithms (e.g., CFS – Completely Fair Scheduler) on threads, not processes.

Real-life: A company (process) with multiple employees (threads). If the company is a black-box to outside (user-level), outsiders only see the company, not individuals. If the company registers each employee (kernel-level), outsiders can manage each separately.

Real-Time Scheduling

Real-time systems must guarantee that tasks meet their **deadlines** (e.g., airbag deployment, industrial robots). Two categories:

- **Hard real-time:** Missing a deadline causes catastrophic failure.
- **Soft real-time:** Missing a deadline degrades performance but not fatal.

Real-time scheduling often uses **priority-based preemptive** algorithms with static or dynamic priorities.

Rate-Monotonic Scheduling (RMS)

Assumptions:

- All tasks are periodic (released at fixed intervals).
- Deadline = period (job must complete before next release).
- Tasks are independent, preemptive.
- No context-switch overhead (theoretical model).

Rule: Assign **higher priority to tasks with shorter periods**. This is an optimal static-priority algorithm for such systems.

Example:

Task	Period (ms)	Execution time (ms)
T1	50	10
T2	100	25
T3	200	30

Rate-monotonic priorities: T1 (period 50) highest, T2 (100) middle, T3 (200) lowest.

Schedulability condition (simplified): For a set of n periodic tasks, RMS is feasible if total utilisation $\leq n \times (2^{1/n} - 1)$. For $n=2$, $2 \times (\sqrt{2} - 1) \approx 0.828$. In practice, many systems use this bound.

Limitation: Not optimal for tasks with different deadlines (if deadline < period). Still widely used in safety-critical systems.

Earliest Deadline First (EDF)

Dynamic priority: At any time, the task with the **earliest absolute deadline** runs next.

- **Optimal** for uniprocessors – if any schedule meets deadlines, EDF will also (with preemption).
- **Utilisation** can approach 100% (unlike RMS' s 69% for many tasks).
- **Disadvantage:** Overhead of sorting deadlines; potential for overload causing many missed deadlines (unpredictable).

Example:

Task	Arrival	Execution	Deadline
T1	0	5	10
T2	0	8	12

At time 0: T1 deadline=10, T2=12 → run T1. At time 5: T1 finishes, run T2 until completion at 13, which misses T2' s deadline 12. EDF would have chosen T2 first? Actually need to check – classic example shows EDF optimal but not infallible.

Better example: T1 (0, 4, 8), T2 (0, 6, 12). EDF runs T1 first, then T2 – both meet deadlines. If T2 had earlier deadline, it would run first.

Comparison:

	Rate-Monotonic (RMS)	EDF
Priority	Static (fixed)	Dynamic
Implementation	Simple (fixed priorities)	Needs deadline sorting
Utilisation bound ($n \rightarrow \infty$)	$\approx 69\%$	100%

Overload behaviour	Predictable (low-priority tasks starve first)	Unpredictable (may miss many deadlines)
Use case	Hard real-time, safety-critical	Soft real-time, multimedia

Real-life example:

- **RMS** – A mail carrier with fixed rounds: shorter routes get higher priority.
- **EDF** – A journalist with article deadlines: the article due soonest is written next, even if it arrived later.

Summary

Concept	Key takeaway
Scheduling criteria	CPU utilisation, throughput, turnaround time, waiting time, response time.
Preemptive vs non-preemptive	Preemptive = OS can interrupt; non-preemptive = process yields voluntarily.
FCFS	Simple but suffers from convoy effect.
SJF (SRTF)	Optimal for minimising average waiting time; requires burst time prediction.
Round Robin	Fair, excellent response; quantum size critical.
Priority scheduling	Starvation mitigated by aging.
Multilevel queue	Fixed categories; rigid.
Multilevel feedback queue	Dynamic movement between queues; used in modern OSes.
Thread scheduling	Kernel-level threads scheduled individually; user-level threads only via process.
RMS	Static priorities based on period; utilisation bound ~69%.
EDF	Dynamic priorities based on deadlines; optimal but more overhead.

Understanding CPU scheduling is key to grasping how an OS balances responsiveness, fairness, and throughput. The next chapter dives into the equally critical topic of memory management.

Chapter 4: Process Synchronization

When multiple processes or threads access shared data concurrently, the results can become unpredictable or incorrect. Process synchronization ensures that cooperative processes execute in a controlled manner, preventing race conditions and preserving data integrity. This chapter covers the classic problems, solutions, and system-level tools used to coordinate access to shared resources.

Critical Section Problem

A **critical section** is a segment of code that accesses shared variables or resources (e.g., a file, a global counter) that must not be executed simultaneously by more than one process or thread.

The **critical section problem** is to design a protocol that ensures when one process is inside its critical section, no other process is allowed to enter its own critical section that accesses the same shared resources.

Structure of a typical process (with a critical section):

```
do {  
    // Entry section – request permission to enter critical section  
    // Critical section – access shared variables  
    // Exit section – release permission  
    // Remainder section – non-critical work  
} while (true);
```

Real-life analogy: An ATM machine (critical section). Only one person can use the ATM at a time. People waiting (processes) must coordinate – if two tried to use it simultaneously, the system could double-dispense cash or corrupt balances.

Solution Requirements

A valid solution to the critical-section problem must satisfy **three conditions**:

Requirement	Meaning
Mutual exclusion	If process P_i is executing in its critical section, no other process can be in its critical section.
Progress	If no process is in its critical section and some processes wish to enter, only those not in their remainder sections can participate in deciding who enters next. The selection cannot be postponed indefinitely.
Bounded waiting	After a process makes a request to enter its critical section, there is a bound (limit) on the number of other processes that can enter before that request is granted. No starvation.

Software Solutions: Peterson's Algorithm

Peterson's algorithm (1981) is a classic software-only solution for **two processes** (P_0 and P_1) that satisfies all three requirements. It works even without hardware atomic operations.

Data structures:

- `int turn;` – indicates whose turn it is to enter.
- `bool flag[2];` – each `flag[i]` indicates that process i is ready to enter.

Algorithm for process i (the other process is $j = 1-i$):

```
do {
    flag[i] = true;           // I want to enter
    turn = j;                 // Let the other go first (courtesy)
    while (flag[j] && turn == j); // Busy wait if other is ready and
    it's its turn

    // Critical section

    flag[i] = false;          // I'm done

    // Remainder section
} while (true);
```

Why it works:

- **Mutual exclusion:** If both processes want to enter, `turn` can be either 0 or 1, so only one proceeds.
- **Progress:** If one process is in its remainder section, `flag[j]` is false, so the other enters immediately.
- **Bounded waiting:** Each process waits at most one turn for the other.

Real-life analogy: Two people sharing a single phone booth. They each raise a hand (`flag[i]=true`) and politely say “You go first” (`turn = j`). If both raise their hands, the one whose turn it is (according to the last spoken “you first”) steps back and lets the other go.

Note: Peterson’s algorithm is elegant but does not work on modern CPU architectures with out-of-order execution unless memory barriers are used. It remains a pedagogical classic.

Hardware Support: Test-and-Set and Compare-and-Swap

Modern processors provide atomic hardware instructions that read, modify, and write memory in one uninterruptible step. These instructions are used to build efficient synchronization primitives.

Test-and-Set (TAS)

Atomically reads a memory location and writes a `true` (or 1) into it.

Pseudo-code:

```
bool test_and_set(bool *target) {  
    bool old = *target;  
    *target = true;  
    return old;  
}
```

Mutual exclusion using TAS (shared variable `lock = false` initially):

```
do {
    while (test_and_set(&lock)); // busy wait
    // Critical section
    lock = false;
    // Remainder section
} while (true);
```

Compare-and-Swap (CAS)

Atomically compares a variable with a given value; if equal, it swaps with a new value.

```
int compare_and_swap(int *value, int expected, int new_value) {
    int old = *value;
    if (old == expected)
        *value = new_value;
    return old;
}
```

Using CAS for mutual exclusion:

```
do {
    while (compare_and_swap(&lock, 0, 1) != 0); // wait until lock
    becomes 0
    // Critical section
    lock = 0;
    // Remainder section
} while (true);
```

Real-life analogy: A restroom key. TAS is like grabbing the key if it's available and simultaneously marking it "taken" in one motion. CAS is like saying "If the key is on the hook, take it and leave a 'busy' sign; otherwise, do nothing."

Mutex Locks and Spinlocks

A **mutex** (MUTual EXclusion) is a simple locking mechanism that supports `acquire()` and `release()` operations.

- **Mutex lock (blocking):** If a thread cannot acquire the lock, it is put to sleep (context switch). Good for long critical sections.
- **Spinlock:** If a thread cannot acquire the lock, it busy-waits (spins) in a loop, consuming CPU. Good for short critical sections (no context-switch overhead).

Implementation using test-and-set:

```
void acquire(lock) {
    while (test_and_set(&lock->flag)); // spin
}
void release(lock) {
    lock->flag = false;
}
```

Feature	Mutex (blocking)	Spinlock
Waiting behaviour	Thread sleeps	Thread spins (busy-wait)
CPU usage during wait	Almost zero	High (wastes cycles)
Context switch overhead	Yes (to scheduler and back)	None
Best when	Critical section long or I/O involved	Critical section very short

Real-life analogy:

- **Mutex:** Waiting for a meeting room – you go to your office (sleep) and get notified when free.
- **Spinlock:** Waiting for a red light – you keep the engine running, tapping the steering wheel.

Semaphores

A **semaphore** is an integer variable that, apart from initialisation, can only be accessed via two atomic operations: `wait()` (also called P, down, decrement) and `signal()` (V, up, increment).

Binary Semaphore

Takes only values 0 and 1 – behaves like a mutex lock.

```
wait(S) {  
    while (S <= 0); // busy wait (or block)  
    S--;  
}  
signal(S) {  
    S++;  
}
```

Counting Semaphore

Can take any non-negative integer: represents the number of available resources.

- `wait(S)` decrements; if `S < 0`, process blocks.
- `signal(S)` increments; if there are blocked processes, one is woken.

Implementation with blocking (to avoid busy waiting):

```

typedef struct {
    int value;
    struct process *waiting_queue;
} semaphore;

void wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        // add this process to waiting_queue
        block(); // OS puts process to sleep
    }
}

void signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        // remove a process from waiting_queue
        wakeup(); // OS makes it ready
    }
}

```

Example: Three printers. Initialize semaphore `printers = 3`. Each process does `wait(&printers)` before printing, `signal(&printers)` after. The fourth process will block until one printer becomes free.

Real-life analogy: Parking garage with 50 spaces. A counter shows available spots. `wait()` = car enters (counter--). `signal()` = car leaves (counter++). If the counter reaches zero, cars must queue (block) outside.

Classical Synchronization Problems

These classic problems illustrate the challenges of synchronization and the use of semaphores, mutexes, and monitors.

1. Bounded Buffer (Producer-Consumer)

Problem: A fixed-size buffer. Producers write data into it; consumers take data out. The producer must not write into a full buffer; the consumer must not read from an empty buffer. No two producers (or consumers) should access the buffer simultaneously.

Solution using semaphores:

```
semaphore mutex = 1;    // protects buffer access
semaphore empty = n;    // number of empty slots
semaphore full = 0;     // number of filled slots

// Producer
do {
    // produce an item
    wait(&empty);        // wait if no empty slot
    wait(&mutex);        // enter critical section
    // add item to buffer
    signal(&mutex);      // leave critical section
    signal(&full);       // increment filled count
} while (true);

// Consumer
do {
    wait(&full);         // wait if nothing to consume
    wait(&mutex);
    // remove item from buffer
    signal(&mutex);
    signal(&empty);      // now one more empty slot
    // consume the item
} while (true);
```

Real-life: A pizza delivery counter (buffer). The baker (producer) makes pizzas and puts them on a shelf. The cashier (consumer) takes them for customers. The baker stops if the shelf is full (`empty=0`). The cashier waits if the shelf is empty (`full=0`). Only one person touches the shelf at a time (`mutex`).

2. Readers-Writers

Problem: A shared database. Multiple readers can read simultaneously, but a writer needs exclusive access. Variations:

- **First readers-writers problem** (no reader starvation): Readers have priority; writers may starve.
- **Second readers-writers problem** (writer priority): Writers have priority; readers may starve.

Solution (first, with reader priority):

```
semaphore rw_mutex = 1;    // writer access
semaphore mutex = 1;       // protects read_count
int read_count = 0;

// Writer
do {
    wait(&rw_mutex);
    // write to database
    signal(&rw_mutex);
} while (true);

// Reader
do {
    wait(&mutex);
    read_count++;
    if (read_count == 1)
        wait(&rw_mutex);    // first reader locks writer out
    signal(&mutex);

    // reading performed

    wait(&mutex);
    read_count--;
    if (read_count == 0)
        signal(&rw_mutex); // last reader unlocks writer
    signal(&mutex);
} while (true);
```

Real-life: A library book. Many people can read the same book simultaneously (readers). But only one person can update the book (writer) – and while updating, no one should read a half-written version.

3. Dining Philosophers

Problem: Five philosophers sit around a table with five chopsticks (or forks). Each philosopher needs two chopsticks to eat – the one on the left and the one on the right. They alternate between thinking and eating. The challenge is to avoid deadlock and starvation.

Naïve solution (leads to deadlock if all grab left chopstick simultaneously).

Solution using semaphores (with deadlock avoidance: pick up both chopsticks only if both free):

```
semaphore chopstick[5] = {1,1,1,1,1};
semaphore mutex = 1;    // to prevent deadlock (allow only 4 philosophers
                        // at table)

void philosopher(int i) {
    while (true) {
        think();
        wait(&mutex);
        wait(&chopstick[i]);
        wait(&chopstick[(i+1)%5]);
        signal(&mutex);
        eat();
        signal(&chopstick[i]);
        signal(&chopstick[(i+1)%5]);
    }
}
```

Other solutions: limit 4 philosophers at a time, asymmetric (odd pick left then right, even right then left), or use a monitor.

Real-life: A group of friends sharing a communal bowl of rice and spoons. Each needs two spoons to serve themselves. Coordination prevents everyone holding one spoon forever (deadlock).

4. Sleeping Barber

Problem: A barber shop with one barber, one barber chair, and a number of waiting chairs. If no customers, the barber sleeps. A customer arriving wakes the barber if asleep. If the barber is busy, the customer waits in a waiting chair if available; otherwise leaves.

Solution (skeleton using semaphores):

```
semaphore customers = 0;    // waiting customers
semaphore barber = 0;      // barber ready
semaphore mutex = 1;       // protects waiting count
int waiting = 0;           // number waiting
int CHAIRS = 5;

// Barber process
do {
    wait(&customers);       // sleep if no customers
    wait(&mutex);
    waiting--;              // one waiting enters
    signal(&barber);        // barber is ready
    signal(&mutex);
    // cut hair
} while (true);

// Customer process
wait(&mutex);
if (waiting < CHAIRS) {
    waiting++;
    signal(&customers);     // wake barber if needed
    signal(&mutex);
    wait(&barber);          // wait for barber
    // get haircut
} else {
    signal(&mutex);         // shop full, leave
}
```

Real-life: Exactly the described barber shop, with the barber napping between customers.

Monitors and Condition Variables

A **monitor** is a high-level abstraction that provides a clean, structured way to achieve synchronization. It is a programming language construct (e.g., in Java, C#) or a library concept (e.g., `pthread_cond` in C).

- The monitor **encapsulates** shared variables and the operations that can access them.
- Only **one thread** at a time can be active inside a monitor (automatic mutual exclusion).
- **Condition variables** allow a thread to wait inside the monitor for a certain condition to become true, temporarily releasing the monitor lock.

Condition variable operations:

- `wait(cond, mutex)` : release mutex, block on cond, re-acquire mutex when signalled.
- `signal(cond)` : wake one waiting thread (if any).
- `broadcast(cond)` : wake all waiting threads.

Example: Bounded buffer using a monitor (pseudo-code):

```

monitor BoundedBuffer {
    int buffer[N];
    int count = 0, in = 0, out = 0;
    condition notFull, notEmpty;

    void produce(int item) {
        while (count == N)
            wait(notFull);
        buffer[in] = item;
        in = (in+1) % N;
        count++;
        signal(notEmpty);
    }

    int consume() {
        while (count == 0)
            wait(notEmpty);
        int item = buffer[out];
        out = (out+1) % N;
        count--;
        signal(notFull);
        return item;
    }
}

```

Real-life analogy: A restroom with a single key (monitor lock). If the restroom is occupied, you wait on a “vacant” condition. When the occupant leaves, they signal – one waiting person enters.

Synchronization in POSIX Threads (Pthreads)

POSIX threads (pthreads) provide a standard API for multithreading on Unix-like systems. Common synchronization primitives:

Mutexes

```
#include <pthread.h>

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

// In threads
pthread_mutex_lock(&mutex);
// critical section
pthread_mutex_unlock(&mutex);
```

`pthread_mutex_trylock()` attempts non-blocking lock acquisition.

Condition Variables

Used together with a mutex:

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;

// Thread that waits for a condition
pthread_mutex_lock(&mutex);
while (condition_not_met) {
    pthread_cond_wait(&cond, &mutex);
}
// condition met, do work
pthread_mutex_unlock(&mutex);

// Thread that signals the condition
pthread_mutex_lock(&mutex);
// change state so condition becomes true
pthread_cond_signal(&cond); // wake one waiting thread
pthread_mutex_unlock(&mutex);
```

- `pthread_cond_broadcast()` wakes all waiting threads.
- Always use a **while loop** (not `if`) with `pthread_cond_wait()` to handle spurious wakeups.

Example: Producer-Consumer with pthreads

```

#define N 10
int buffer[N];
int in = 0, out = 0, count = 0;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t notFull = PTHREAD_COND_INITIALIZER;
pthread_cond_t notEmpty = PTHREAD_COND_INITIALIZER;

void *producer(void *arg) {
    int item;
    while (1) {
        item = produce_item();
        pthread_mutex_lock(&mutex);
        while (count == N)
            pthread_cond_wait(&notFull, &mutex);
        buffer[in] = item;
        in = (in+1) % N;
        count++;
        pthread_cond_signal(&notEmpty);
        pthread_mutex_unlock(&mutex);
    }
}

```

Real-life: A ticket booking system – the mutex prevents double booking; condition variables let a thread wait for a seat to become available.

Summary

Concept	Key takeaway
Critical section	Code segment accessing shared resources that must be mutually exclusive.
Requirements	Mutual exclusion, progress, bounded waiting.
Peterson' s algorithm	Software solution for two processes (pedagogical, not widely used in practice).

Hardware support	TAS, CAS – atomic instructions that simplify lock implementation.
Mutex / Spinlock	Mutex blocks, spinlock busy-waits. Choose based on critical section length.
Semaphores	Counting semaphores manage resources; binary semaphores act as mutexes.
Classical problems	Bounded buffer, readers-writers, dining philosophers, sleeping barber.
Monitors	Language construct with automatic mutual exclusion and condition variables.
Pthreads	Standard Unix threading API with <code>pthread_mutex</code> and <code>pthread_cond</code> .

Understanding process synchronization is crucial to writing correct concurrent programs. The next chapter explores deadlock – what happens when synchronization goes wrong.

Chapter 5: Deadlocks

In a multiprogramming environment, processes compete for resources. Sometimes, a set of processes gets stuck forever, each waiting for a resource held by another in the set. This situation is a **deadlock**. This chapter explains how deadlocks occur, how to represent them, and the classic strategies to prevent, avoid, detect, or ignore them.

Deadlock Characterization

A deadlock occurs when two or more processes are each waiting indefinitely for an event (usually resource release) that can only be caused by another waiting process. For a deadlock to happen, **four necessary conditions** must hold simultaneously.

The Four Conditions

Condition	Description
Mutual exclusion	At least one resource must be held in non-shareable mode. Only one process can use the resource at a time.
Hold and wait	A process holding at least one resource is waiting to acquire additional resources held by other processes.
No preemption	Resources cannot be forcibly taken away from a process. Only the holding process can release them voluntarily.
Circular wait	There exists a set $\{P_0, P_1, \dots, P_n\}$ such that P_0 waits for a resource held by P_1 , P_1 waits for P_2 , ..., P_n waits for P_0 .

All four must hold **at the same time**. Break any single one and deadlock becomes impossible – this is exactly what the prevention strategies later in the chapter do.

Real-life analogy: Two cars meet at a four-way stop intersection.

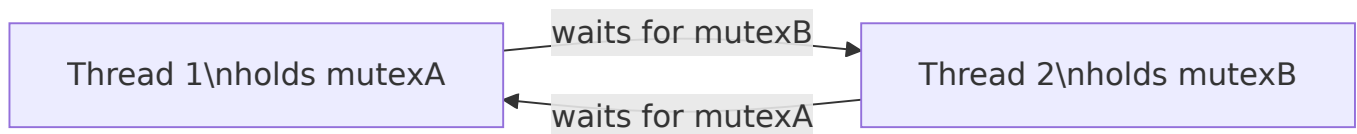
- **Mutual exclusion** – Only one car can occupy the intersection at a time.
- **Hold and wait** – Each car holds its position (already in the intersection or at the line) and waits for the other to move.
- **No preemption** – No traffic officer can push a car aside.

- **Circular wait** – Car A waits for Car B to clear the intersection; Car B waits for Car A to clear. Both block.

The same thing in code – the classic two-lock deadlock. Each thread grabs one mutex, then reaches for the other:

```
// Thread 1                // Thread 2
lock(&mutexA);              lock(&mutexB);
lock(&mutexB);  // blocks   lock(&mutexA);  // blocks
// critical section        // critical section
unlock(&mutexB);            unlock(&mutexA);
unlock(&mutexA);            unlock(&mutexB);
```

If Thread 1 acquires `mutexA` and Thread 2 acquires `mutexB` before either reaches its second `lock()`, both block forever. Note that this program may run correctly thousands of times – deadlock only appears when the interleaving is just wrong, which is what makes these bugs so hard to reproduce.



That closed loop of arrows is the circular wait condition. Every deadlock, no matter how many processes are involved, reduces to a cycle like this one.

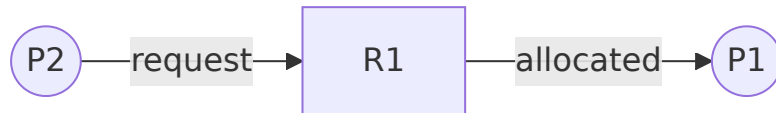
Resource Allocation Graph

A **Resource Allocation Graph (RAG)** is a directed graph that visually represents the state of resource allocation. It helps detect whether deadlock exists.

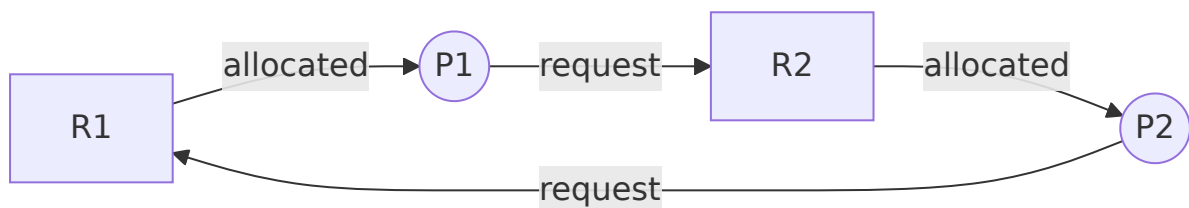
- **Vertices:** Two types
 - Process vertices (circles): $P_1, P_2, \dots, P_n$
 - Resource vertices (squares): $R_1, R_2, \dots, R_m$ (each may have one or more identical instances)
- **Edges:**
 - **Request edge** ($P_i \rightarrow R_j$): Process P_i has requested an instance of R_j and is waiting.
 - **Assignment edge** ($R_j \rightarrow P_i$): An instance of R_j has been allocated to P_i .

The direction of the arrow is the whole trick: **towards** a resource means “I am asking for this” , **away from** a resource means “I already have this” .

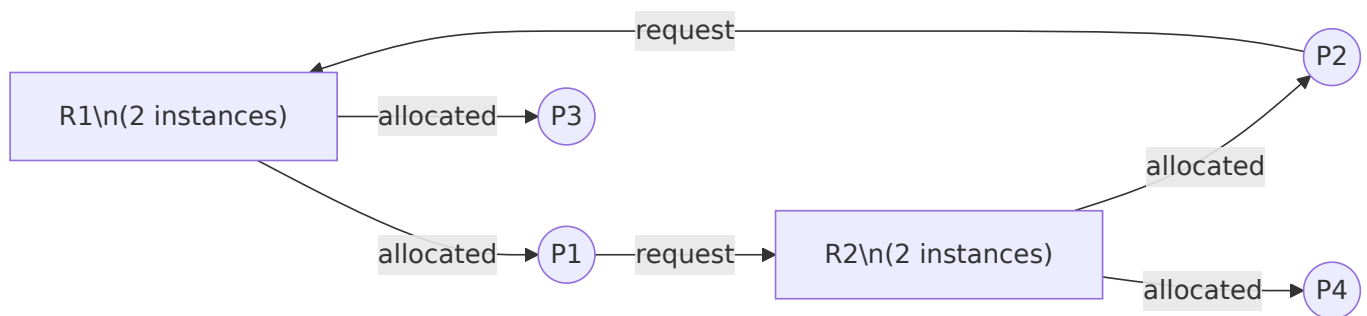
Case 1 – No cycle, no deadlock. P1 holds R1 and wants nothing else; P2 waits for R1. As soon as P1 finishes, R1 frees up and P2 proceeds.



Case 2 – Cycle with single-instance resources → deadlock. P1 holds R1 and requests R2; P2 holds R2 and requests R1. Neither can move.



Case 3 – Cycle with multiple instances → not necessarily deadlock. Here R1 and R2 each have **two** instances, and all four are allocated. There is a cycle P1 → R2 → P2 → R1 → P1, yet no deadlock: the spare instances are held by P3 and P4, neither of which is waiting for anything. When P4 finishes it releases its instance of R2, P1 takes it, and the cycle unravels.



Rules for reading a RAG:

Graph shape	Conclusion
No cycle	No deadlock , guaranteed.
Cycle, all resources have a single instance	Deadlock , guaranteed.
Cycle, some resource has multiple instances	Possible deadlock – must check further (run the detection algorithm).

In short: a cycle is a *necessary* condition for deadlock, but only *sufficient* when every resource in the cycle has exactly one instance.

Real-life analogy: A whiteboard showing who holds which resource (e.g., printer, scanner) and who is waiting for what. An arrow from a person to a resource means “I want that” ; an arrow from resource to person means “this person has it” . If you see a circle of arrows, people may be deadlocked.

Methods for Handling Deadlocks

Operating systems use one of four approaches to deal with deadlocks, ranging from idealistic to pragmatic.

Approach	Philosophy	Overhead	Practicality
Prevention	Ensure at least one condition never holds	High	Rarely used
Avoidance	Allocate resources carefully using extra info	Medium	Feasible for fixed resources
Detection & Recovery	Allow deadlock, then break it	Medium	Common in databases
Ignorance (Ostrich)	Assume deadlocks won' t happen	None	Most general-purpose OSes (Windows, Linux)

1. Deadlock Prevention

Prevention ensures that **at least one** of the four necessary conditions is impossible. The OS designs resource allocation to break conditions.

Breaking Mutual Exclusion – Not generally possible. Some resources are inherently non-shareable (e.g., printer). But for read-only files, shared access can be allowed.

Breaking Hold and Wait – Require a process to request **all** resources it will ever need at once, or release all held resources before requesting more.

- **Drawback:** Low resource utilisation; starvation possible.

Breaking No Preemption – If a process holding some resources requests another that is unavailable, the OS forcibly takes away the currently held resources (preempts) and makes the process restart.

- **Drawback:** Difficult to implement if resources have state (e.g., a partially written file).

Breaking Circular Wait – Impose a **total ordering** of all resource types. Each process must request resources in increasing order (e.g., tape drive (1) then printer (2) then disk (3)). A process requesting a higher-numbered resource must already hold all lower-numbered ones.

- **Practical:** Used in some systems. For example, the `lock_order` rules in the Linux kernel.

Real-life: A restaurant that eliminates circular wait by assigning numbers to tables: you must eat at table 1 before table 2, and never go from table 3 back to table 2.

2. Deadlock Avoidance (Banker's Algorithm)

The **Banker's Algorithm** (Dijkstra, 1965) avoids deadlock by simulating resource allocation before granting it. The OS requires each process to declare its **maximum** resource needs in advance.

Key concept – Safe state: A state is safe if there exists a sequence of processes such that each process, when given its maximum remaining need, can finish with the currently available resources plus resources held by previously finished processes.

If granting a request leads to an **unsafe state**, the OS delays the request (process waits).

Data structures (for n processes, m resource types):

- `Available[m]` : available instances of each resource.
- `Max[n][m]` : maximum demand of each process.
- `Allocation[n][m]` : currently allocated to each process.
- `Need[n][m] = Max[i][j] - Allocation[i][j]` : remaining need.

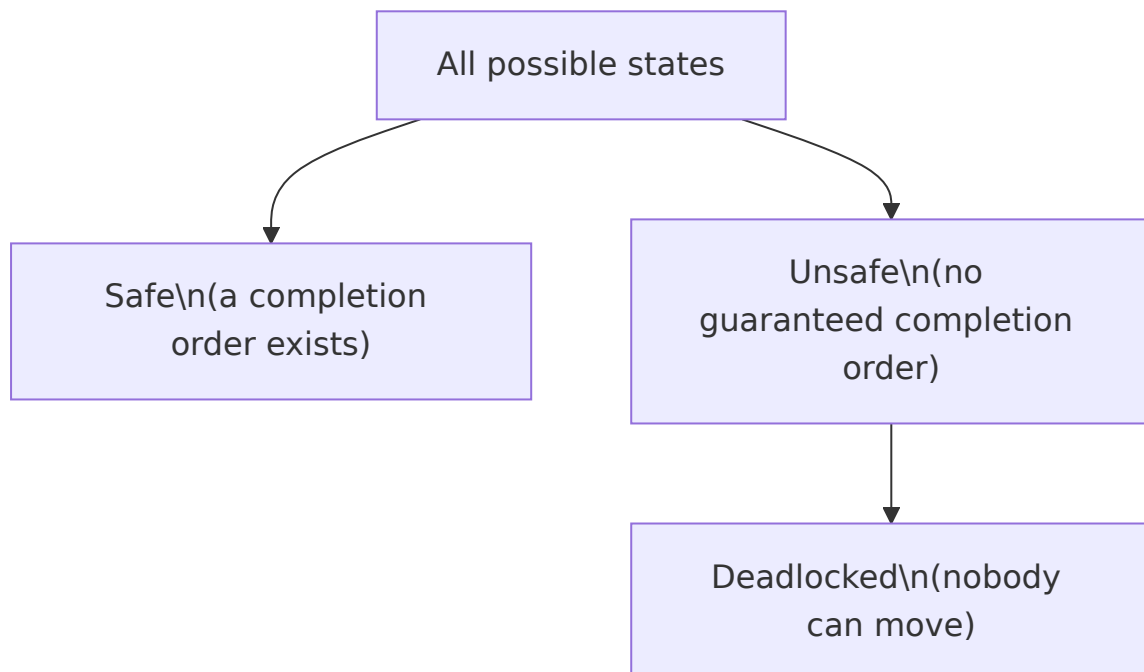
Safety Algorithm (find if safe state exists):

1. `Work = Available`, `Finish[n] = false`.
2. Find an i such that `Finish[i]==false` and `Need[i] <= Work`.
3. `Work = Work + Allocation[i]`, `Finish[i]=true`. Go to step 2.
4. If all `Finish[i]==true`, state is safe.

Resource-Request Algorithm for process P_i requesting $Request[i]$:

1. If $Request[i] > Need[i]$, error (exceeded maximum).
2. If $Request[i] > Available$, wait.
3. Pretend to allocate: $Available -= Request[i]$, $Allocation[i] += Request[i]$, $Need[i] -= Request[i]$.
4. Run Safety Algorithm. If safe, allocate for real; else roll back and P_i waits.

Safe, unsafe, and deadlocked states – these are not the same thing. Every deadlocked state is unsafe, but an unsafe state is not yet a deadlock; it is a state from which the system *might* drift into one. The Banker’ s algorithm is conservative: it refuses to leave the safe region at all.



Worked Example 1 – Single Resource Type

A system has **12 instances** of one resource type.

Process	Allocation	Max	Need (Max – Allocation)
P0	5	10	5
P1	2	4	2
P2	2	9	7

Allocated = $5 + 2 + 2 = 9$, so **Available** = $12 - 9 = 3$.

Run the safety algorithm with **Work** = 3 :

Step	Candidate	Need $\leq$ Work?	Work after it finishes
1	P1	$2 \leq 3$ ✓	$3 + 2 = 5$
2	P0	$5 \leq 5$ ✓	$5 + 5 = 10$
3	P2	$7 \leq 10$ ✓	$10 + 2 = 12$

All processes finish, so the state is **safe** with safety sequence **(P1, P0, P2)**.

Notice the order matters. Starting with P0 (needs $5 > 3$) or P2 (needs $7 > 3$) is impossible – only P1 can run first, and it is P1 returning its resources that unblocks everyone else.

Now suppose P2 requests one more instance. Tentatively grant it: **Allocation[P2]** = 3 , **Need[P2]** = 6 , **Available** = 2 .

Step	Candidate	Need $\leq$ Work?	Work after it finishes
1	P1	$2 \leq 2$ ✓	$2 + 2 = 4$
2	P0	$5 \leq 4$ ✗	—
2	P2	$6 \leq 4$ ✗	—

After P1 finishes, only 4 instances are free but both remaining processes need more than that. No safety sequence exists, so this state is **unsafe**. The Banker's algorithm therefore **denies P2's request** and makes it wait – even though the resource was physically available.

Worked Example 2 – Multiple Resource Types

The standard textbook case: five processes and three resource types with totals **A = 10, B = 5, C = 7**.

Process	Allocation (A B C)	Max (A B C)	Need (A B C)
P0	0 1 0	7 5 3	7 4 3

P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1

Total allocated = (7, 2, 5), so **Available** = (10, 5, 7) – (7, 2, 5) = (3, 3, 2).

Step	Candidate	Need $\leq$ Work?	Work after it finishes
1	P1	(1,2,2) $\leq$ (3,3,2) ✓	(3,3,2) + (2,0,0) = (5,3,2)
2	P3	(0,1,1) $\leq$ (5,3,2) ✓	(5,3,2) + (2,1,1) = (7,4,3)
3	P4	(4,3,1) $\leq$ (7,4,3) ✓	(7,4,3) + (0,0,2) = (7,4,5)
4	P2	(6,0,0) $\leq$ (7,4,5) ✓	(7,4,5) + (3,0,2) = (10,4,7)
5	P0	(7,4,3) $\leq$ (10,4,7) ✓	(10,4,7) + (0,1,0) = (10,5,7)

The state is **safe** with sequence **(P1, P3, P4, P2, P0)**. A comparison like (1, 2, 2) $\leq$ (3, 3, 2) must hold for **every** resource type – a single component exceeding Work disqualifies the process for that round.

Exam tip: safety sequences are not unique. (P1, P3, P4, P0, P2) also works here. Any valid sequence proves the state is safe, so if a question asks "is this state safe?", finding one sequence is enough.

Limitations:

- Fixed number of processes/resources.
- Processes must declare maximum needs – often impossible.
- Overhead of $O(m \times n^2)$ on each request.

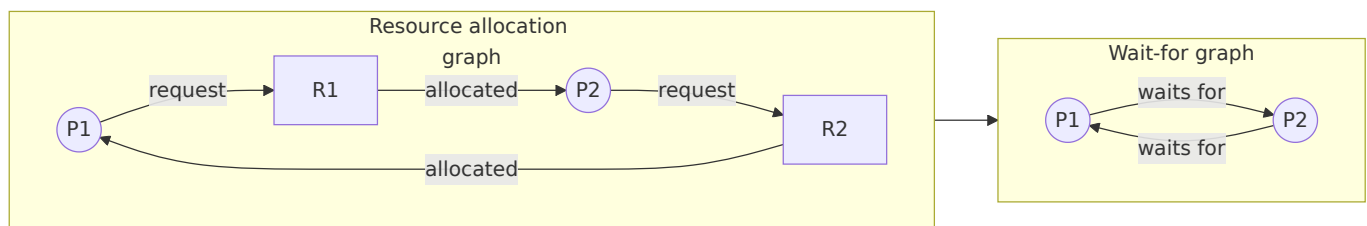
Real-life: A bank (OS) that never lends money if it cannot guarantee to satisfy all future customer withdrawal requests. The banker knows each customer's credit limit.

3. Deadlock Detection and Recovery

Allow deadlock to occur, detect it, and then break it. Used in systems where deadlock is rare or where processes are long-running.

Detection with Single Instance per Resource – Use the **Wait-for Graph**: a reduced version of the resource allocation graph where resource nodes are removed; an edge $P_i \rightarrow P_j$ means P_i waits for a resource held by P_j . Periodic graph cycle detection (e.g., depth-first search) identifies deadlock.

To build it, collapse each pair of edges $P_i \rightarrow R_k \rightarrow P_j$ into a single edge $P_i \rightarrow P_j$:



The cycle survives the reduction, so this system is deadlocked. Cycle detection costs $O(n^2)$ for n processes.

Detection with Multiple Instances – Use an algorithm similar to Banker's but without **Max** – just look for processes that can finish with current available resources. If none can finish, deadlock exists.

How often should detection run? This is a real trade-off. Running it on every resource request catches deadlock the instant it forms and identifies exactly which process caused it, but the overhead is severe. Running it once an hour is cheap, yet several cycles may exist by then and it becomes impossible to tell which process to blame. A common compromise is to trigger detection only when CPU utilisation drops below a threshold (say 40%), since a system with many blocked processes tends to go idle.

Recovery Options:

Method	Description	Problem
Process termination	Abort all deadlocked processes (or one by one)	Expensive; losing work.
Resource preemption	Take resources from a process and give to another	Need to rollback process to safe checkpoint, possible starvation.

In practice, many systems opt to simply restart the system or kill processes manually.

Real-life: A system administrator who notices all servers are stuck – kills a process (the “sledgehammer”) or restarts a service.

4. Deadlock Ignorance (Ostrich Algorithm)

The **Ostrich Algorithm** assumes deadlocks are so rare that the overhead of prevention, avoidance, or detection is not worth the cost. The OS does nothing to handle deadlocks.

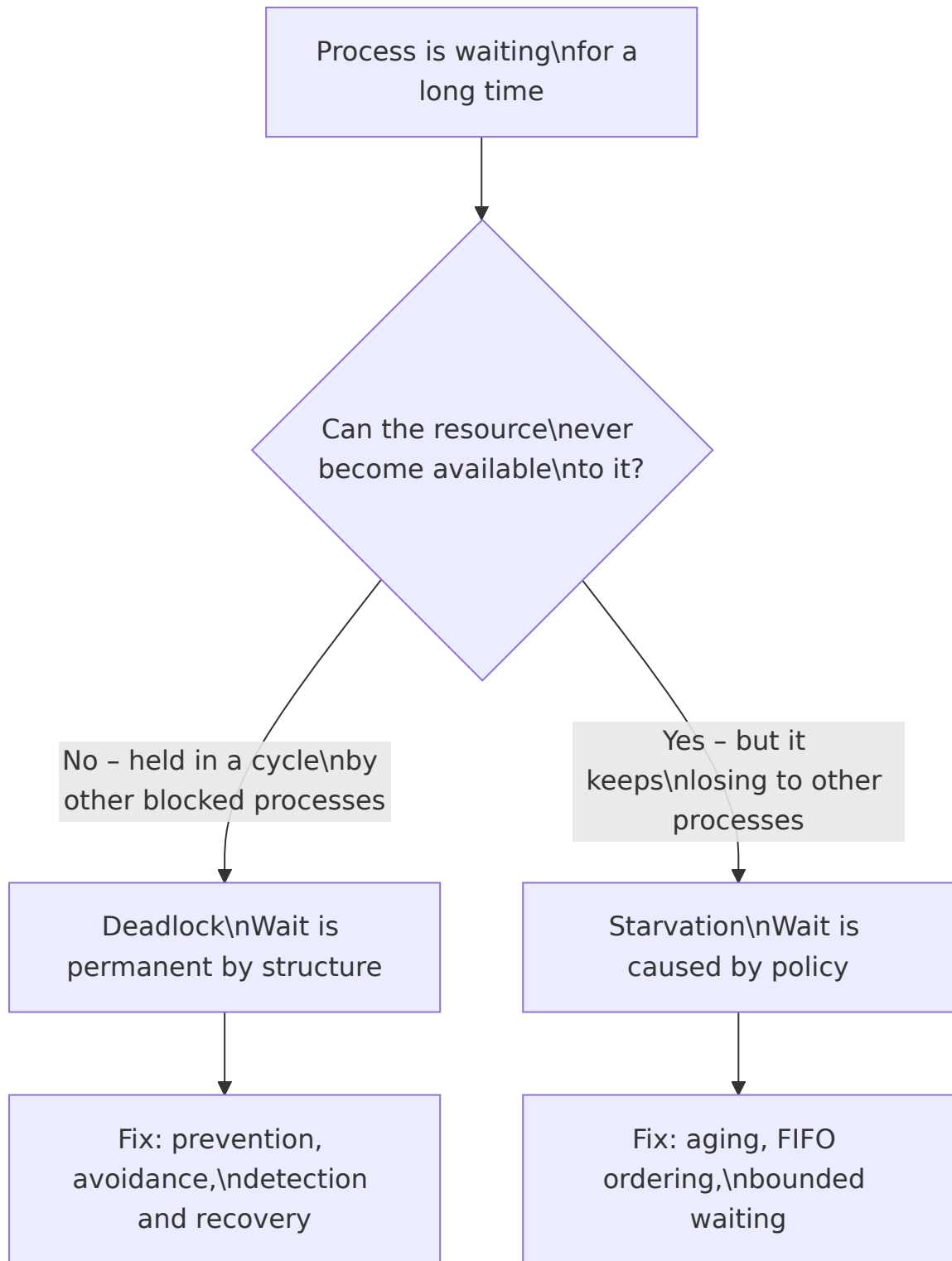
- Used in **most general-purpose OSes** (Windows, Linux, macOS) for most resource types.
- Why? Deadlocks typically occur due to programmer bugs, not OS design. The OS provides locking primitives; it’s the programmer’s responsibility to avoid circular wait.
- If a deadlock happens, the system freezes or crashes – user reboots.

Real-life: Putting your head in the sand. The ostrich ignores the problem. For many applications (e.g., your web browser deadlocking – you just restart it), this is acceptable.

Deadlock vs Starvation

Both leave a process waiting for a long time, which is why they are so easily confused. The difference is whether the wait can *ever* end on its own.

In a **deadlock**, the processes involved are blocked permanently. Nothing in the system can change that – the resources they need are held by each other, so no amount of waiting helps. In **starvation** (also called *indefinite postponement*), the process is technically still eligible to run. Resources do become free, but the scheduler keeps handing them to someone else. A starving process could be served at any moment; it just never is.



Aspect	Deadlock	Starvation
Cause	Circular wait – all four necessary conditions hold.	Scheduling or priority policy that keeps favouring other processes.
Processes affected	A <i>set</i> of processes, blocking each other.	Usually a <i>single</i> low-priority process.

Resources	Held by the blocked processes themselves; never released.	Are released regularly, but always allocated to someone else.
Can it resolve on its own?	Never – outside intervention is required.	Yes – if the load drops or priorities shift, the process may finally run.
CPU utilisation	Involved processes are blocked; utilisation may drop sharply.	System stays busy – it is doing useful work, just not for the victim.
Also known as	Circular wait, deadly embrace.	Indefinite postponement, indefinite blocking.
Typical remedy	Prevention, avoidance (Banker's), detection and recovery.	Aging – gradually raise the priority of a process the longer it waits.
Relationship	Deadlock always implies its processes are starved of resources.	Starvation does not imply deadlock.

Real-life analogy: Deadlock is a traffic gridlock where four cars each block the next in a circle – nobody moves again until a traffic officer physically intervenes. Starvation is a driver trying to merge onto a busy motorway during rush hour: the road is moving fine and gaps do appear, but faster cars keep taking them. The driver *could* merge at any moment, and eventually will once traffic thins.

A third case – livelock. Two processes keep responding to each other and changing state, but make no forward progress. Unlike deadlock, they are not blocked – they are actively burning CPU. The classic analogy is two people meeting in a corridor who both step aside in the same direction, over and over. Deadlock is a traffic jam; livelock is a polite standoff.

	Deadlock	Livelock	Starvation
Process state	Blocked	Running	Ready / waiting
CPU consumed	None	Wasted on retries	None (for the victim)
Resolves itself?	No	Rarely (needs randomised backoff)	Possibly

Summary

Concept	Key takeaway
Deadlock conditions	Mutual exclusion, hold and wait, no preemption, circular wait (all four needed).
Resource allocation graph	Visual tool to represent processes, resources, requests, and allocations.
Cycles in a RAG	No cycle → no deadlock. Cycle + single instances → deadlock. Cycle + multiple instances → maybe.
Deadlock prevention	Break one of the four conditions (e.g., resource ordering, request all at once).
Deadlock avoidance	Banker' s algorithm – ensure system stays in safe state; requires max needs.
Safe vs unsafe state	Safe → a completion order exists. Unsafe → no guarantee, but not yet a deadlock.
Deadlock detection	Periodic check (wait-for graph or detection algorithm); then recover by termination or preemption.
Deadlock ignorance	Do nothing – used by Windows, Linux for most resources; user reboots.
Deadlock vs starvation	Deadlock never resolves without intervention; starvation may resolve, and is cured by aging.

Understanding deadlocks closes the loop on concurrency control. The next chapter moves to the physical side of the OS: memory management.

Chapter 6: Memory Management

Memory is the physical heart of a computer system. The operating system must allocate memory to processes efficiently, protect one process from another, and create the illusion of a large, private address space. This chapter explains the hardware and OS techniques that make modern memory management possible.

Memory Hierarchy

Not all memory is equal. A typical computer has a hierarchy of storage, from fast and expensive to slow and cheap.

Level	Technology	Typical size	Access time	Managed by
Registers	CPU internal	Few hundred bytes	0.2-0.5 ns	Compiler
Cache (L1, L2, L3)	SRAM	32KB – 32MB	1-10 ns	Hardware
Main memory (RAM)	DRAM	4GB – 512GB	50-100 ns	OS
Swap space (disk)	SSD/HDD	100GB – 2TB	0.1-10 ms	OS
File storage	SSD/HDD	256GB – multiple TB	ms – seconds	User/OS

The OS’ s memory manager hides this complexity, making main memory appear as a uniform, linear array of bytes, and optionally extending it to disk (virtual memory, covered in Chapter 7).

Real-life analogy: A desk (registers) holds what you are working on right now. A bookshelf nearby (cache) holds reference books. A filing cabinet (RAM) holds current projects. A warehouse (disk) holds archived files. You move data up the hierarchy when actively needed.

Address Binding

Programs reference memory using addresses. The binding of these symbolic addresses to actual physical memory locations can happen at different times.



Three Binding Times

Binding time	When	Example	Pros / Cons
Compile time	Addresses fixed at compilation	Absolute code (e.g., MS-DOS .COM)	Fast but inflexible; program must load at a specific address.
Load time	Relocation by loader at program load	Relocatable code (e.g., .exe with base address)	Flexible; however, once loaded, address cannot move.
Execution time	Binding delayed until runtime; uses MMU	Modern OSes (Linux, Windows)	Maximum flexibility; allows swapping and paging.

Real-life analogy:

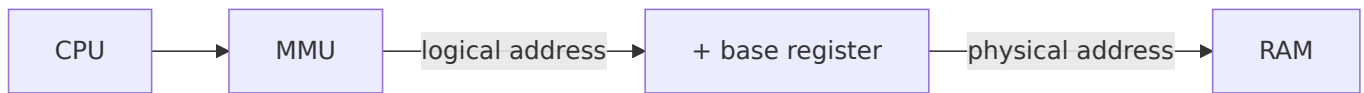
- **Compile time:** A train timetable fixed to exact platform numbers.
- **Load time:** A conference seating plan – seats assigned when you arrive, but you cannot move later.
- **Execution time:** A taxi – your pickup location is determined when you call (runtime).

Logical vs Physical Address Space

- **Logical address** (virtual address): Generated by the CPU while a program executes. The program sees only this address space.
- **Physical address:** The actual location in RAM hardware. The OS and MMU translate logical to physical addresses.

Memory Management Unit (MMU): Hardware that performs runtime address translation. For example, a simple MMU uses a **base (relocation) register**:

```
physical address = logical address + base register
```



With a bounds (limit) register, the OS can protect processes: if `logical address >= limit`, trap to OS.

Real-life analogy: A hotel room key labelled “Room 412” (logical address). The hotel’s internal map (MMU) translates that to actual physical room 1012 on the third floor. You never need to know the physical location.

Memory Allocation

When a process is created, the OS must allocate memory for its code, data, and stack.

Contiguous Allocation

Each process occupies a single contiguous block of physical memory.

Fixed Partitions (Static)

Memory is divided into fixed-size regions at boot time. A process is loaded into a partition large enough to hold it.

- **Pros:** Simple, no external fragmentation.
- **Cons:** Internal fragmentation (wasted space inside a partition), limited number of active processes.

Dynamic Partitions (Variable)

Partitions are created dynamically to exactly match a process’s size. The OS maintains a list of free holes.

Placement algorithms for selecting a hole:

- **First-fit:** Use the first hole large enough. Fast, generally good.
- **Best-fit:** Use the smallest hole that fits. Tends to create tiny useless holes.
- **Worst-fit:** Use the largest hole. Leaves larger remaining holes.

Real-life: Parking on a busy street (dynamic partitions). First-fit: take the first spot you see. Best-fit: measure every spot to find the one that exactly fits your car – you waste time.

Fragmentation

Type	Description	Cause
Internal fragmentation	Wasted space inside an allocated partition. Example: fixed partition size 8MB, process needs 5MB → 3MB wasted.	Fixed partitions; also page alignment in paging.
External fragmentation	Total free memory is sufficient to satisfy a request, but no single contiguous block is large enough.	Dynamic partitioning – holes become scattered.

Compaction: Shuffle memory contents to combine small holes into one large hole. Only possible with dynamic (execution-time) binding. Expensive because it requires moving processes.

After compaction

Process A

Process B

Process C

Free 23KB

Before compaction

Process A

Free 10KB

Process B

Free 5KB

Process C



Paging

Paging solves external fragmentation by dividing physical memory into fixed-size blocks called **frames** (typically 4KB) and logical memory into same-size blocks called **pages**. A process' s pages can be scattered across non-contiguous frames.

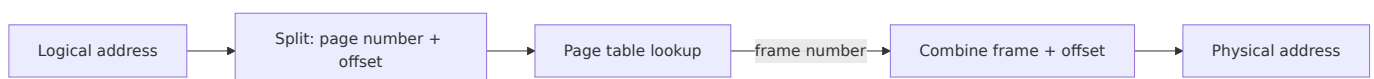
Key idea: Logical address = (page number, page offset). The OS maps page number to frame number via a **page table**.

Address Translation

Given logical address with m bits, page size 2^n bytes:

- **Page number (p)** = top $(m - n)$ bits.
- **Page offset (d)** = lower n bits.

Example: 32-bit address, 4KB pages (2^{12}). Page number = bits 31-12, offset = bits 11-0.



Page Table Entry (PTE)

Each PTE typically contains:

Field	Meaning
Frame number	Physical frame where the page resides.
Valid (present) bit	Page is in memory (1) or swapped to disk (0).
Dirty (modified) bit	Page has been written to (needs write-back to disk on eviction).
Reference (accessed) bit	Page has been read or written (used for page replacement algorithms).

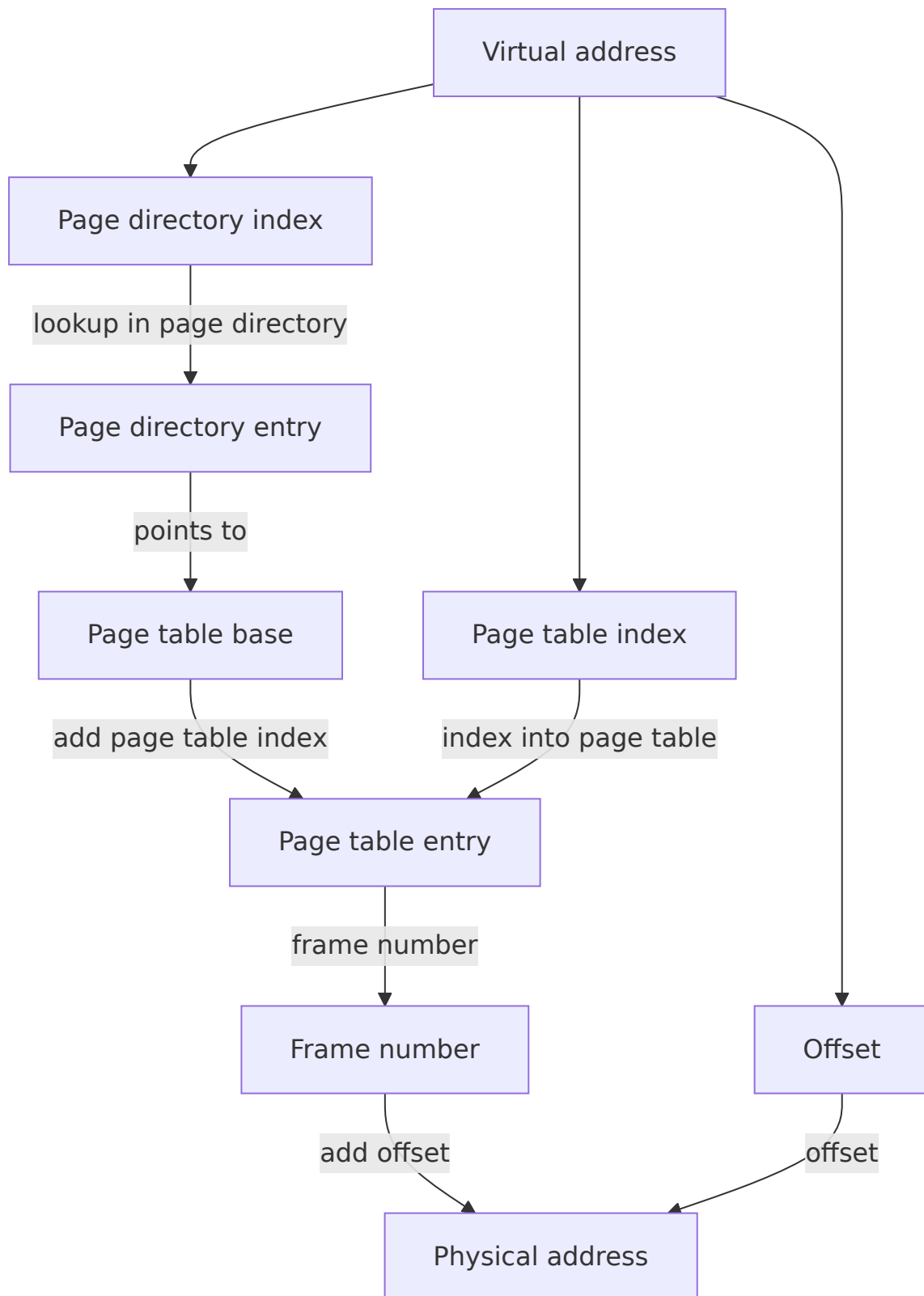
Protection bits	Read, write, execute permissions.
-----------------	-----------------------------------

Multi-level Page Tables

A single page table for a 64-bit address space would be impossibly large (2^{64} / page size entries). Multi-level page tables reduce memory overhead by only allocating levels that are actually used.

Example: 2-level page table on x86-32 (10+10+12 bits). Virtual address split:

- Level 1 (page directory index): 10 bits → selects a page directory entry pointing to a page table.
- Level 2 (page table index): 10 bits → selects a PTE in that table.
- Offset: 12 bits.



Advantage: Sparse address spaces waste very little memory – only directory and the page tables that are actually used.

Inverted Page Table

Instead of one page table per process, an **inverted page table** has one entry per physical frame. Each entry stores (process ID, page number) mapping. Used in PowerPC, UltraSPARC,

and some IBM systems.

- **Size:** (physical memory size / frame size) entries – fixed and small.
- **Lookup:** Hashing the (PID, page number) pair to find the entry. Slower than multi-level tables.

Translation Lookaside Buffer (TLB)

Page tables are stored in RAM. Accessing memory would require two RAM accesses (page table + actual data) – too slow. The TLB is a **hardware cache** that stores recently used page-to-frame translations.

TLB hit: Translation found in TLB – single memory access.

TLB miss: Must walk page table (in RAM) – two or more accesses, then cache the mapping.

Typical TLB: 32-512 entries, fully associative or set-associative.

TLB coverage: TLB entries $\times$ page size = maximum memory that can be accessed without page table walk. For 4KB pages and 64 TLB entries = 256KB.

Large pages (e.g., 2MB or 1GB) improve TLB coverage and are used for memory-intensive workloads (databases, virtual machines).

Context switch and TLB: When switching processes, the TLB contains entries belonging to the old process. Solutions:

- Flush TLB (costly).
- Tagged TLB: each entry includes an Address Space ID (ASID) to differentiate processes.

Real-life analogy:

- **Paging** = A filing system where each file (page) can be stored in any drawer (frame), with an index (page table).
- **TLB** = A small sticky note on your desk listing the most-used file locations, so you don't need to open the big index every time.

Segmentation

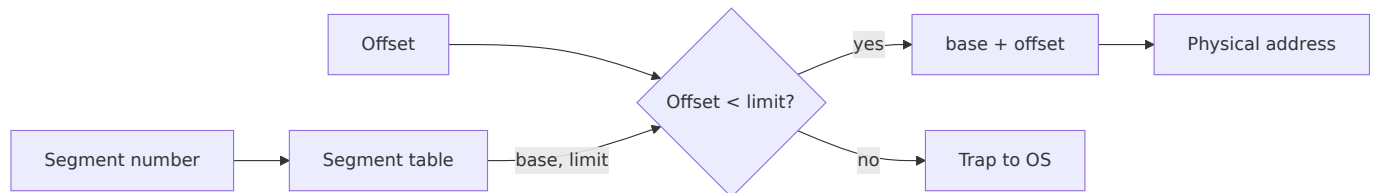
Segmentation gives the programmer a view of memory as consisting of **variable-sized segments** – code, data, stack, heap, etc. – each with its own base and limit.

Segment Table

Each process has a segment table. A logical address is `<segment - number , offset>` . The segment table entry contains:

- **Base:** Starting physical address of the segment.
- **Limit:** Length of the segment.

The MMU checks that `offset < limit` , then computes `physical = base + offset` .



Advantages:

- Segments grow and shrink separately (e.g., stack expansion).
- Protection per segment (code=read-only, data=read-write).
- Sharing: two processes can share a code segment by referencing the same base/limit.

Disadvantage: External fragmentation – segments are contiguous variable-sized blocks.

Segmentation with Paging (Intel x86)

The x86 architecture (and many others) combines segmentation and paging: segmentation provides multiple logical address spaces, and each segment is divided into pages.

Process:

1. Logical address → `<selector, offset>` .
2. Segment descriptor (from Global Descriptor Table or Local Descriptor Table) gives base and limit of the segment. The OS may set segment base to 0 to effectively disable segmentation.
3. The resulting linear address (formerly called virtual address) is then translated to physical address via paging (multi-level page tables).

This two-step approach is complex but gives flexibility. Most modern OSes (Linux, Windows) use a **flat memory model**: one segment (or a few) covering the entire address space, relying almost entirely on paging for protection and translation.

Real-life analogy:

- **Segmentation alone** = You have separate boxes (code box, data box, stack box) of different sizes. You can rearrange the boxes in a warehouse.

- **Segmentation + paging** = You have boxes, but each box is further divided into smaller standard-size containers (pages) that can be stored in different shelves in the warehouse.

Summary

Concept	Key takeaway
Memory hierarchy	Registers → cache → RAM → swap → storage; trade-off speed vs size.
Address binding	Can be compile time, load time, or execution time (most modern).
Logical vs physical	Logical = program's view; physical = real RAM; MMU translates.
Contiguous allocation	Fixed or dynamic partitions; suffers fragmentation.
Fragmentation	Internal (wasted inside block) and external (scattered holes).
Paging	Fixed-size pages and frames; eliminates external fragmentation; page table maps pages to frames.
Page table entries	Contain frame number, valid, dirty, reference bits.
Multi-level page tables	Reduce memory overhead for sparse address spaces.
Inverted page table	One entry per physical frame; used in some architectures.
TLB	Hardware cache for address translations; critical for performance.
Segmentation	Variable-sized segments (code, data, stack) with protection and sharing.
x86 combination	Segmentation provides base/limit, then paging for fine-grain translation.

Understanding memory management is fundamental to paging, swapping, and virtual memory – the subject of the next chapter.

Chapter 7: Virtual Memory

Virtual memory is a technique that separates the logical address space of a process from physical memory. It allows programs to be larger than physical RAM, enables efficient sharing, and simplifies memory management. This chapter explains how virtual memory works, how the OS handles page faults, and the algorithms used to decide which pages stay in memory.

Demand Paging

Demand paging loads pages into physical memory only when they are accessed (on demand), not when the process starts. This reduces memory usage and speeds up process startup.

Under demand paging, the page table has a **valid-invalid bit** per entry:

- **Valid (1):** The page is in physical memory (frame allocated).
- **Invalid (0):** The page is not in memory – either never loaded or swapped out to disk (swap space).

When the program tries to access an invalid page, the hardware traps to the OS – this is a **page fault**.

Lazy swapper (pager): The OS that only brings pages into memory when needed. Compare to a swapper that moves whole processes.

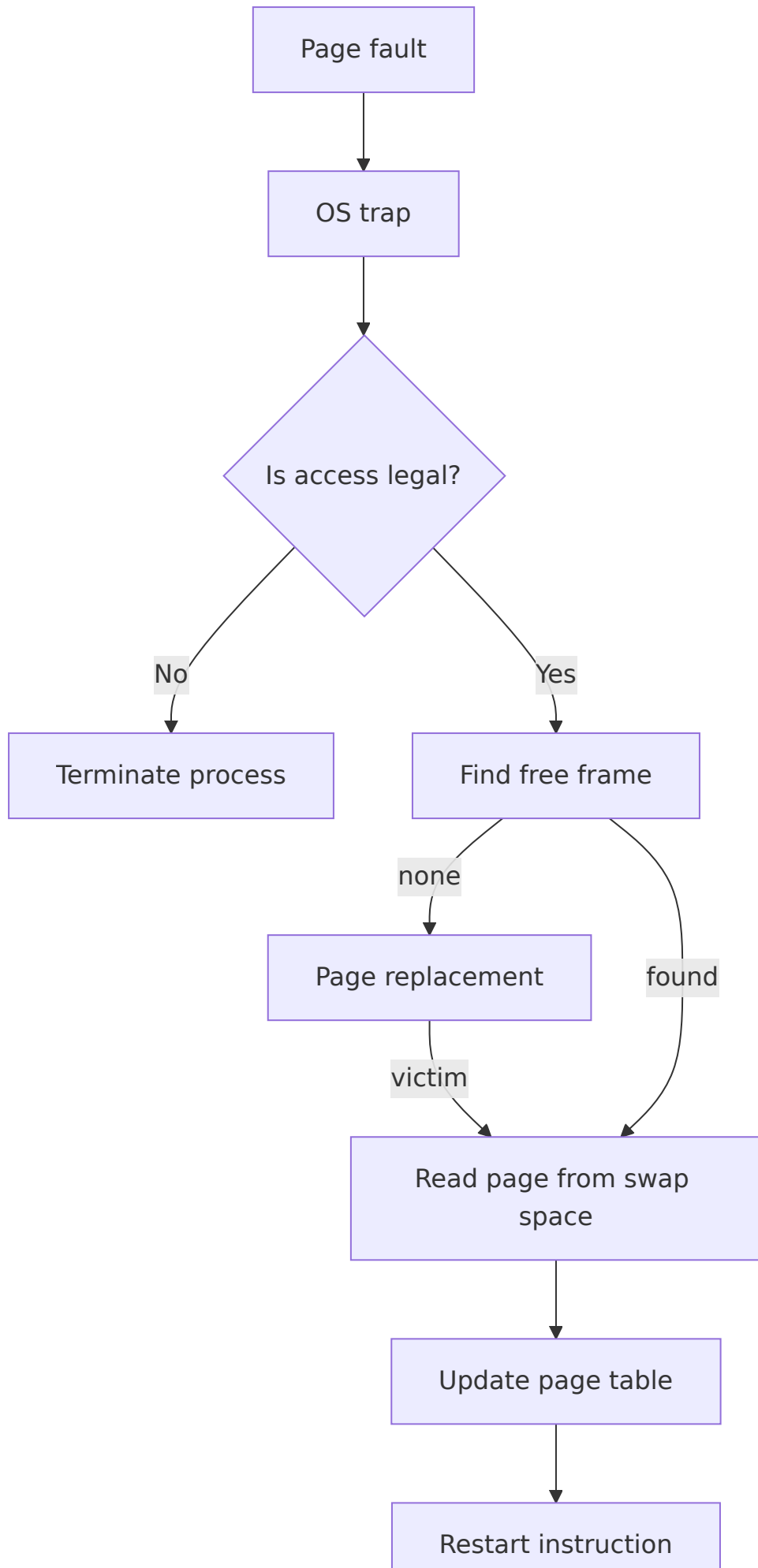
Real-life analogy: A university library that does not bring every book from storage to the reading room at once. Only when you request a book (page fault) does a librarian fetch it from the basement.

Page Fault Handling

Steps taken when a page fault occurs:

1. The hardware traps to the OS kernel, saving the program counter and process state.
2. The OS determines which virtual page was accessed (from the faulting address).

3. The OS checks if the access is legal (within address space, correct permissions). If illegal, the process is terminated (segmentation fault).
4. The OS finds a free physical frame. If none, a page replacement algorithm selects a victim frame (Chapter 7.4).
5. The OS schedules a disk read to bring the required page from swap space into the chosen frame.
6. While the disk I/O is in progress, the process is blocked. The OS may context-switch to another process.
7. When the disk I/O completes, the OS updates the page table entry: valid bit = 1, frame number set.
8. The OS restarts the faulting instruction, which now successfully accesses the page.



Performance of demand paging: Effective access time = $(1 - p) \times \text{memory access time} + p \times \text{page fault time}$. Even a tiny page fault rate ($p \approx 0.0001$) can slow the system significantly because disk access is $\sim 10^6$ times slower than RAM.

Copy-on-Write (COW)

The **fork()** system call traditionally copied the entire parent address space to the child – expensive and slow. **Copy-on-write** defers copying: both parent and child share the same physical pages marked as read-only. When either tries to write to a page, a page fault occurs, and the OS makes a private copy of that page for the writing process.

- **Advantage:** Very fast fork. Unmodified pages are never copied.
- **Use case:** Common after fork, the child immediately calls `exec()` – no copying at all.
- **Implementation:** Page table entries have a COW flag. On write fault, copy the page, update both page tables to read-write (or keep original read-only).

Real-life analogy: A teacher gives the same textbook to two students. As long as neither writes in it, they share. When one wants to underline a sentence, that student gets a photocopy of that page (copy-on-write).

Page Replacement Algorithms

When a page fault occurs and there is no free frame, the OS must **evict** a page from memory. Page replacement algorithms aim to choose the page that will cause the fewest future faults.

1. FIFO (First-In, First-Out)

Evict the page that has been in memory the longest. Implemented as a circular queue.

- **Advantage:** Simple.
- **Disadvantage:** Poor performance; suffers from **Belady's anomaly** – increasing number of frames can increase page faults.

Belady's anomaly example for FIFO (reference string: 1,2,3,4,1,2,5,1,2,3,4,5):

- 3 frames $\rightarrow$ 9 faults.
- 4 frames $\rightarrow$ 10 faults (worse!).

2. Optimal Algorithm (MIN, OPT, or Belady's)

Evict the page that will not be used for the **longest time** in the future. This is provably optimal for minimizing page faults.

- **Advantage:** Best possible performance (lower bound).
- **Disadvantage:** Impossible to implement in practice – requires knowledge of the future. Used as a benchmark to compare other algorithms.

3. LRU (Least Recently Used)

Evict the page that has not been used for the longest time (based on past behaviour). This approximates OPT by assuming that pages used recently will be used again soon (temporal locality).

Implementation approaches:

- **Counters:** Each page table entry has a last-use time field; at each memory access, update the time. On eviction, scan for smallest time. Expensive.
- **Stack:** Maintain a stack of page references; move accessed page to top. Bottom page is LRU. Hardware support needed for fast updates.

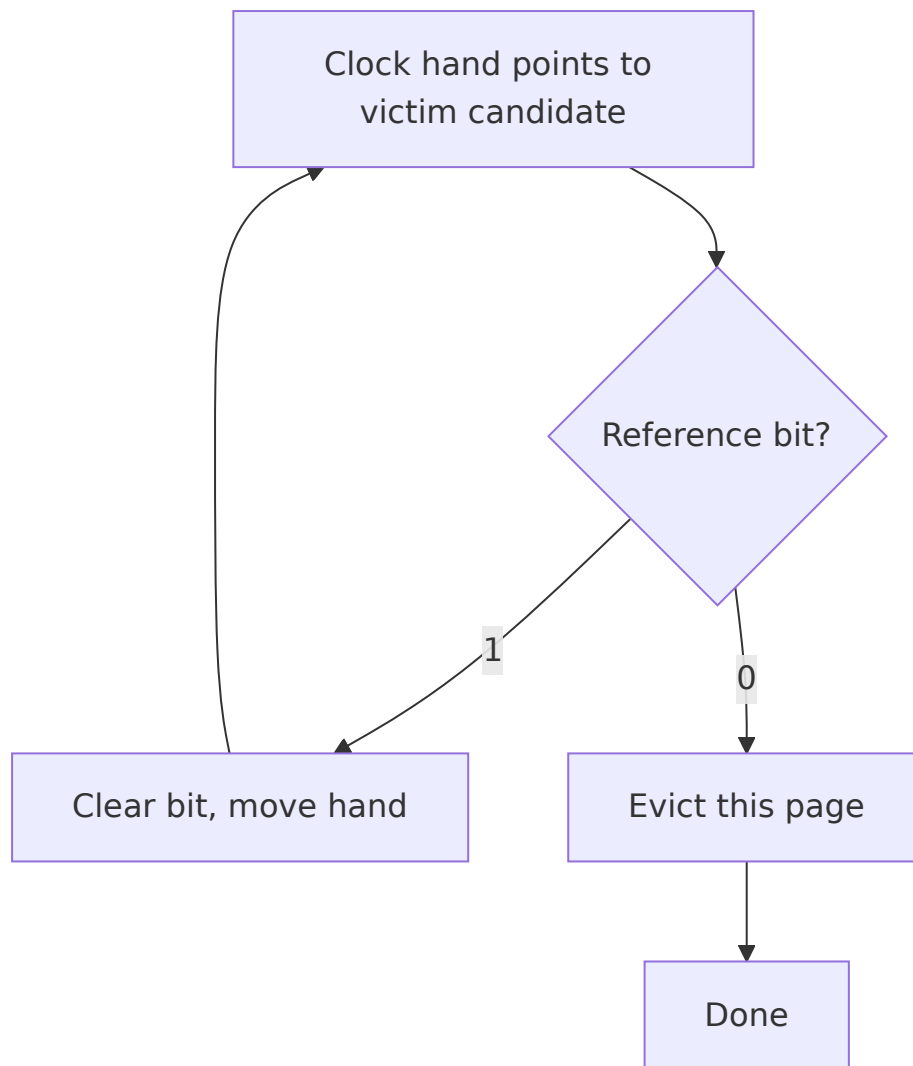
LRU does not suffer from Belady's anomaly; it is a stack algorithm.

4. Clock (Second-Chance) Algorithm

A practical approximation of LRU that uses a **reference bit** (accessed bit) per page. Pages are arranged in a circular list; a hand pointer moves around.

- If the page's reference bit = 1, clear it (give a second chance) and move hand.
- If the page's reference bit = 0, evict it.

Enhanced Clock (NUR – Not Used Recently): Use both reference bit and dirty bit. Prefer to evict clean (not dirty) pages to avoid writing to disk.



5. LFU (Least Frequently Used) and MFU (Most Frequently Used)

- **LFU**: Evict the page with the smallest access count. Assumes pages used many times will be used again.
- **MFU**: Evict the page with the largest access count. Assumes pages with high count are already cached and will not be needed soon.

Both suffer from implementation overhead (counters can grow large). Also, LFU can retain pages that were heavily used early but never again (no aging). A modified version (LFU with aging) decays counters over time.

Comparison of algorithms:

Algorithm	Implementation complexity	Performance	Works in practice?
-----------	---------------------------	-------------	--------------------

FIFO	Very low	Poor, Belady anomaly	Only for very simple systems
Optimal	Not feasible	Best (theoretical)	Benchmark only
LRU	High (hardware or costly)	Very good	Yes, if hardware support
Clock	Low (software with reference bit)	Good (close to LRU)	Most common in OSes
LFU	Medium	Fair (can cache obsolete pages)	Rare

Real-life analogy:

- **FIFO** = A queue at a ticket counter; the person who arrived first is served first, even if they are buying a complicated ticket.
- **LRU** = A bookshelf where you put books you read back on the left; the rightmost book has been untouched longest.
- **Clock** = A security guard who walks around a building, checking each room's "recently used" light; if light is on, he turns it off and moves on; if off, he evicts the room.

Thrashing and Working Set Model

Thrashing occurs when a process spends more time **page faulting** than executing. The disk becomes the bottleneck; CPU utilisation drops dramatically.

Cause: The process does not have enough physical frames to hold its **locality** – the set of pages it actively uses. The OS repeatedly evicts pages that will be immediately needed again.

Working Set Model (Denning, 1968): The working set of a process at time t is the set of pages referenced in the last Δ references (or time interval). If a process's working set size exceeds available frames, thrashing occurs.

Strategies to prevent thrashing:

- **Working set algorithm:** Keep track of each process's working set; only allow a process to run if its working set fits in memory.

- **Page Fault Frequency (PFF):** Set upper and lower bounds on allowed page fault rates. If a process faults too often, give it more frames; if fault rate is very low, take frames away.
- **Suspend a process:** If the sum of working sets exceeds total memory, swap out a process entirely.

Real-life analogy: A chef with only 2 sq ft of counter space trying to prepare a 10-ingredient dish. He constantly moves pots on and off the stove (thrashing) and gets little cooking done. The solution: give him more counter space (frames) or reduce the recipe (working set).

Page Size Selection

Choosing the page size involves trade-offs. Most systems use 4 KB, 8 KB, or 16 KB pages (or larger, e.g., 2 MB “huge pages”).

Factor	Small pages	Large pages
Internal fragmentation	Less (waste at page boundary)	More (last page of process)
Page table size	Larger (more entries)	Smaller
I/O overhead	More transfers	Fewer, bigger transfers
Locality	Better (waste less on rarely used parts)	Worse (brings in extra unused data)
TLB coverage	Lower (fixed number of TLB entries)	Higher (each entry maps more memory)

Modern OSes support multiple page sizes (superpages, huge pages) to allow applications (e.g., databases, VMs) to request larger pages for better TLB performance.

TLB Reach and Coverage

TLB reach = (number of TLB entries) × (page size). This is the amount of memory that can be mapped by the TLB at once. If a program’s active working set fits within the TLB reach, TLB misses are rare.

Example: $64 \text{ TLB entries} \times 4 \text{ KB page} = 256 \text{ KB TLB reach}$. A database with a 10 MB working set would thrash the TLB.

Solutions:

- Increase page size (e.g., 2 MB huge pages $\rightarrow 64 \text{ entries} \times 2 \text{ MB} = 128 \text{ MB reach}$).
- Increase TLB size (more entries, expensive in hardware).
- Use multiple TLB levels (L1 and L2 TLBs).

TLB coverage refers to the proportion of memory accesses handled without a page table walk. High coverage improves performance.

Memory-Mapped Files

Memory-mapped files allow a file on disk to be mapped directly into a process' s virtual address space. Reads and writes to that memory region are automatically reflected in the file (with page caching).

How it works:

1. The OS sets up page table entries for the mapped range.
2. Initially, all pages are marked invalid (or demand-paged).
3. When the process accesses a page, a page fault occurs.
4. The OS reads the corresponding file block from disk into a physical frame.
5. If the page is modified (dirty), the OS writes it back to the file when the page is evicted or the file is unmapped.

System calls: `mmap()` (Unix), `CreateFileMapping()` + `MapViewOfFile()` (Windows).

Advantages:

- Simplifies file I/O – treat file as memory (no explicit `read` or `write` calls).
- Efficient sharing: multiple processes can map the same file and share pages.
- The OS handles caching and I/O automatically.

Real-life analogy: A shared whiteboard in an office. Each person (process) sees the same board content (file). When someone writes on it (memory write), everyone sees the change when they look. The cleaner (OS) saves the board to a photo (disk) periodically.

Summary

Concept	Key takeaway
Demand paging	Load pages only when accessed; reduces memory usage.
Page fault	Trap when accessing invalid page; OS brings in the page.
Copy-on-write	Share pages between fork()' d processes until a write occurs.
Page replacement	FIFO (simple, Belady anomaly), Optimal (unimplementable), LRU (good but costly), Clock (practical), LFU/MFU (rare).
Thrashing	Excessive paging due to insufficient frames for working set.
Working set model	Active pages of a process; thrashing occurs if working set > available frames.
Page size trade-off	Small pages: less fragmentation, bigger page tables; large pages: better TLB reach, more internal waste.
TLB reach	Entries $\times$ page size; must cover working set for good performance.
Memory-mapped files	Map file into address space; I/O happens automatically on page faults.

Virtual memory completes the memory management picture. The next chapter moves to I/O and file systems, where these principles are applied to storage devices.

Chapter 8: File Systems

A file system is the OS component that manages how data is stored, organised, and retrieved on persistent storage devices (disks, SSDs). This chapter covers file concepts, directory structures, allocation methods, free space management, disk scheduling, and real-world file system designs.

File Concept

A **file** is a named collection of related information stored on a secondary storage device. It is the logical unit of storage, hiding the physical details of the disk.

File Attributes

Attribute	Description
Name	Human-readable identifier (e.g., <code>report.txt</code>).
Identifier	Unique number (inode number) used internally by OS.
Type	Indicates format (e.g., <code>.c</code> , <code>.pdf</code> , executable).
Location	Pointer to where the file is stored on disk.
Size	Current size in bytes (and possibly maximum).
Protection	Access permissions (read, write, execute for owner/group/others).
Timestamps	Creation, last modification, last access times.
User ID / Owner	Which user owns the file.

File Operations

Common system calls for file manipulation:

Operation	Purpose
<code>create</code>	Make a new file (allocate space, add directory entry).

open	Prepare file for use – returns a file descriptor.
read	Transfer bytes from file to memory.
write	Transfer bytes from memory to file.
seek	Move the file pointer (random access).
close	Flush buffers, release resources.
delete	Remove file from disk and directory.
truncate	Discard existing content, keep file.

File Types

Type	Example extensions	Purpose
Regular	.txt , .c , .pdf	User data or programs.
Directory	folder	Contains file names and references.
Special (device)	/dev/sda (Unix)	Represents I/O devices.
Named pipe	FIFO	Inter-process communication.
Socket	/var/run/docker.sock	Network or local IPC.

Real-life analogy: A file is like a physical document in a filing cabinet. Its attributes include the document title (name), the drawer number (location), its size, and who can read it (protection).

Directory Structure

A **directory** is a container that stores file names and their associated metadata (or pointers to metadata). Directories organise files hierarchically.

Single-Level Directory

All files are in one directory. Simple but suffers from name collisions and poor organisation on multiuser systems.

Example: MS-DOS floppy disks (root only).

Two-Level Directory

Separate directory per user: a master file directory (MFD) contains user directories, each user has their own user file directory (UFD). Different users can have files with the same name.

Tree-Structured Directory

A general hierarchy: directories can contain subdirectories. Users can create unlimited nesting. Current working directory simplifies pathnames.

Absolute path: from root (`/home/user/file.txt`).

Relative path: from current directory (`./file.txt`).

Acyclic-Graph Directory

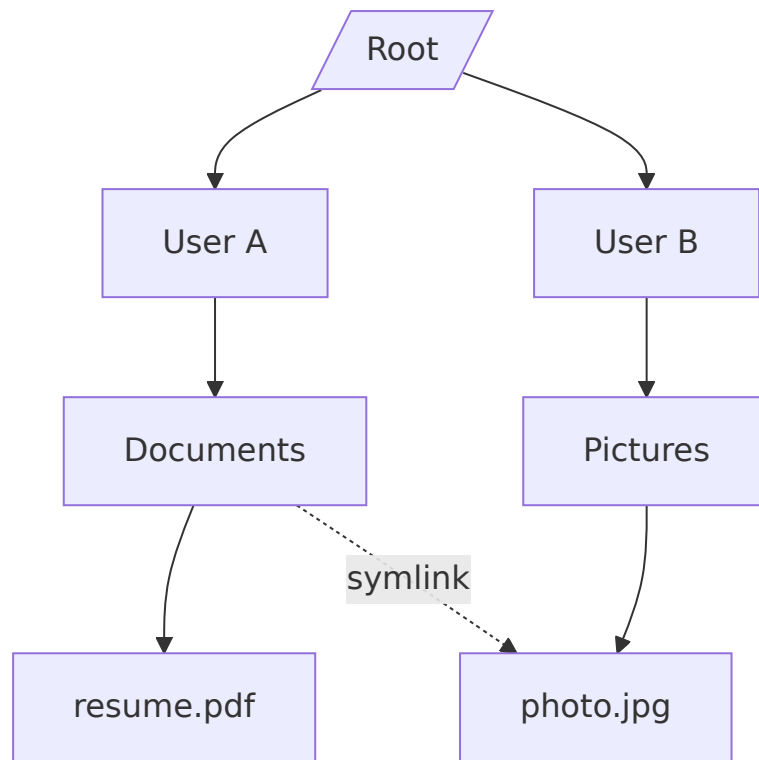
Allows sharing of files or subdirectories via **links** (shortcuts, aliases). Two types:

- **Hard link:** Multiple directory entries point to the same file (same inode) – cannot cross file systems.
- **Soft link (symbolic link):** A special file containing a path to the target – can cross file systems.

Problem: Deleting a file becomes tricky (reference counts needed).

General Graph Directory

Permits cycles (directory containing a subdirectory that links back to its parent). Leads to problems traversing directories (infinite loops). Solutions: garbage collection, refuse to remove files with references, or disallow cycles.



Real-life analogy:

- Single-level = one big drawer with all files.
- Tree-structured = a filing cabinet with folders and subfolders.
- Acyclic graph = multiple folder labels pointing to the same physical file (like sticky notes on a box).

File System Mounting

Mounting is the act of attaching a file system (from a disk partition, USB drive, CD-ROM) to an existing directory path in the main file system tree. The directory where it is attached is the **mount point**.

Steps:

1. The OS checks that the device contains a valid file system (superblock).
2. The OS records the mount point in a mount table.
3. Access to the mount point path is redirected to the root of the mounted file system.

Example (Unix): `mount /dev/sdb1 /mnt/usb`

Real-life: A bookshelf (main file system) with empty slots (mount points). You put a labelled binder (file system from a USB drive) into a slot. When you look at that slot, you see the

contents of the binder.

File Allocation Methods

The OS must decide how to allocate disk blocks to files. Three classic methods.

1. Contiguous Allocation

Each file occupies a set of **contiguous blocks** on disk. The directory entry stores the start block and length.

- **Pros:** Simple, fast sequential and random access (only one seek).
- **Cons:** External fragmentation (holes); file growth is difficult (must pre-allocate or move file).

Real-life: A cinema with reserved seats – all seats in a row. If you need more seats, you must move to a different row.

2. Linked Allocation

Each file is a **linked list** of disk blocks. The directory entry stores the first block pointer. Each block contains a pointer to the next block (or a file allocation table).

- **Pros:** No external fragmentation; file can grow easily.
- **Cons:** Poor random access (must traverse pointers); reliability (one lost pointer ruins the rest).

File Allocation Table (FAT): A separate table (at the start of the partition) holds pointers for all blocks. The directory entry stores the first block number. The FAT entry for each block points to the next. Random access still requires following the chain in memory (but can be cached).

Real-life: A treasure hunt where each clue (block) tells you where the next clue is. Finding the 50th clue requires following 49 previous ones.

3. Indexed Allocation

Each file has its own **index block** – an array of pointers to all the file's data blocks. The directory entry points to the index block.

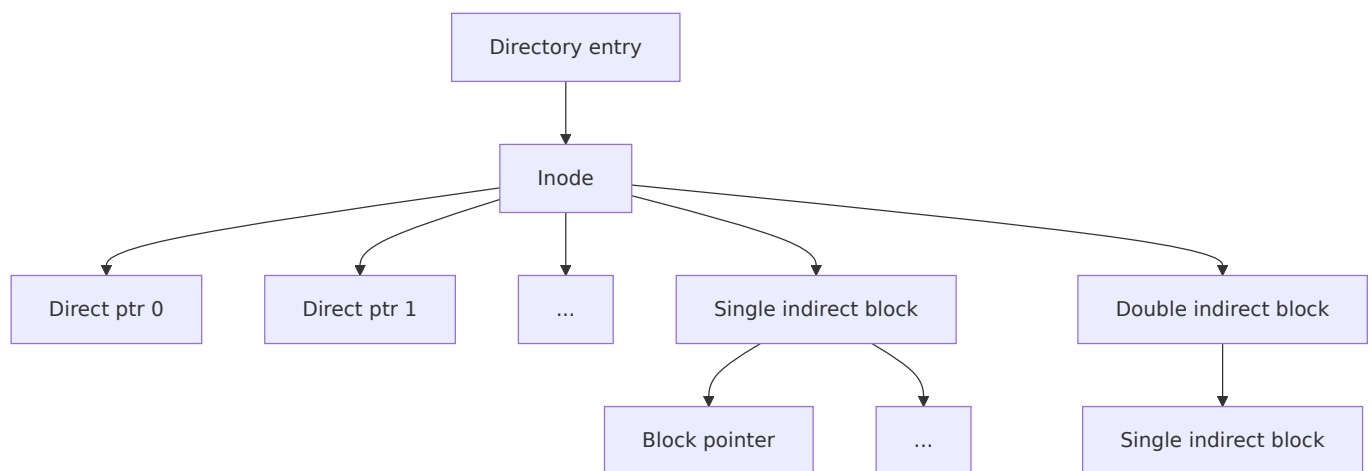
- **Pros:** Direct random access (look up index block, then data block). No external fragmentation.

- **Cons:** Overhead of index block; small files waste space if index block is fixed size (e.g., 512 entries).

Extensions (Unix inode):

- Direct pointers (10-12 pointers).
- Single indirect pointer (points to a block of pointers).
- Double indirect pointer (points to a block of single indirect blocks).
- Triple indirect pointer (for very large files).

This scheme supports huge files with minimal overhead for small files.



Real-life: A library card catalogue. The card (index block) lists where each chapter (data block) is located in the library.

Free Space Management

The OS must track which disk blocks are free and allocate them when needed.

Bit Vector (Bitmap)

One bit per block (1 = free, 0 = allocated). The bit vector fits in memory for moderate-sized disks (e.g., 1 TB with 4 KB blocks: $1 \text{ TB} / 4 \text{ KB} = 2^{28}$ blocks $\rightarrow$ 256 MB of bitmap – large but manageable). Finding a free block is a scan for a 1-bit.

Linked List

Free blocks are linked together. The first free block pointer is stored. Each free block contains a pointer to the next free block.

- **Pros:** Simple, no extra space (except pointer in each free block).

- **Cons:** Traversing the list requires reading blocks (disk I/O). To allocate, need to read one block.

Grouping

Store the addresses of many free blocks in one block. The last free block points to the next group. Reduces traversal reads.

Counting

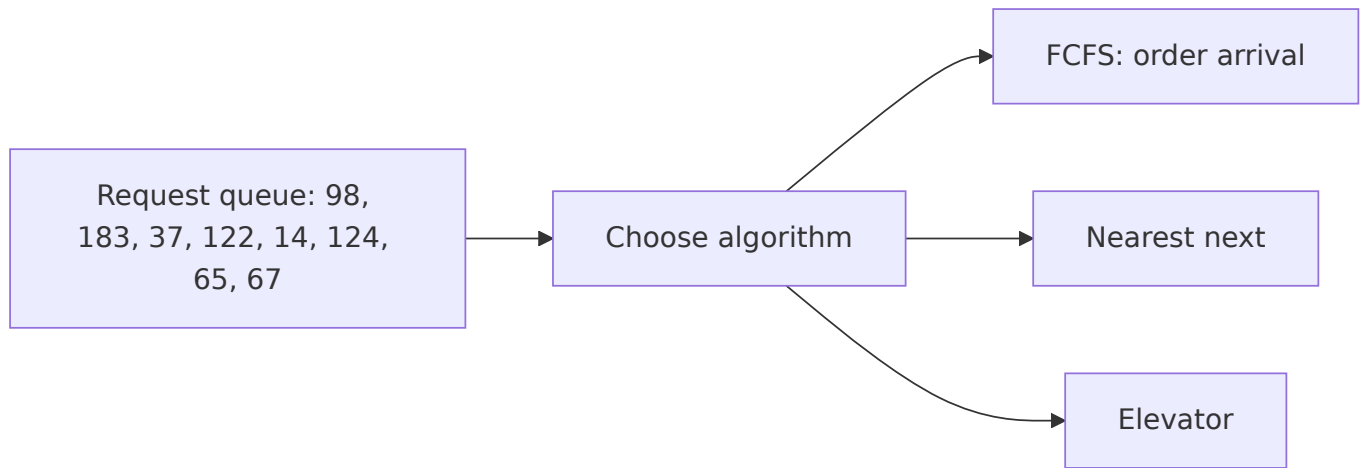
Free blocks often appear in contiguous runs (especially after file deletion). Record addresses of first free block and the count of contiguous free blocks. The list of (address, count) pairs is compact.

Disk Scheduling

The OS (or disk driver) orders read/write requests to minimise seek time and improve throughput.

Common Algorithms

Algorithm	Description	Pros	Cons
FCFS	First-come, first-served	Fair, simple	Poor performance (wild seeks)
SSTF	Shortest seek time first	Better throughput	May starve far-away requests
SCAN (elevator)	Move arm from one end to the other, servicing requests	Fair, no starvation	Requests at ends wait longer
C-SCAN	Circular SCAN – only one direction, then jump back	Uniform waiting time	Extra jump time (no I/O during jump)
LOOK	Like SCAN but stop at last request (no go to end)	Less wasted motion	Still biased
C-LOOK	Like C-SCAN but stop at last request	Best of both	Slightly more complex



Real-life analogy:

- **FCFS:** A elevator that serves people in order of pressing the button (goes up and down wildly).
- **SCAN:** A lift that goes from bottom to top, then top to bottom, stopping at floors with pending requests.
- **LOOK:** Same but doesn't go to the top floor if no one requested it.

File System Layout

A typical Unix-like file system on disk is divided into blocks. Common structure:

Block range	Name	Purpose
Block 0	Boot block	Contains bootstrap loader (code to boot the OS).
Block 1	Superblock	File system metadata: block size, number of blocks, free block count, inode table size, magic number.
Blocks 2..N	Inode table	Array of inodes, each describing a file (metadata + pointers).
Rest	Data blocks	Actual file content and directory entries.

Additional structures:

- **Block bitmap** or **inode bitmap** for free space tracking (often placed after superblock).

- **Journal or log** area (for journaling file systems).

Real-life: A warehouse (disk) with a map (superblock) at the entrance, a catalogue (inode table) giving details of each box, and the shelves (data blocks) where boxes are stored.

Journaling and Log-Structured File Systems

Journaling (Write-ahead logging)

Before modifying file system structures, the OS writes a **journal** (log) of the intended changes. After the journal is safely on disk, the actual changes are made. If a crash occurs, the OS replays the journal to restore consistency.

Journaling levels:

- **Writeback:** Metadata only, data may be inconsistent.
- **Ordered:** Metadata journaled; data written before journal commit.
- **Data:** Full data journaling – safe but slow.

Examples: ext3, ext4 (ordered), NTFS (metadata journaling), XFS.

Real-life: A pilot’s checklist – before takeoff, you write “flaps up” in a log (journal), then do it. If you are interrupted, you can resume from the log.

Log-Structured File System (LFS)

Instead of overwriting files in place, LFS writes all changes sequentially to the end of a circular log (like a tape). The file system index (inode map) is also periodically written. Old blocks are garbage-collected.

- **Pros:** Excellent write performance (sequential writes), fast crash recovery.
- **Cons:** Read requires scanning the log; garbage collection overhead.

Real-life: A journalist’s notebook – every new fact is written at the end, never erasing. An index at the back points to the latest version of each topic.

File System Examples

FAT32 (File Allocation Table)

- **Origins:** MS-DOS, Windows 95/98, still used for USB drives.
- **Structure:** Boot sector, FAT table (two copies), root directory (fixed location), data area.
- **Limitations:** Max file size 4 GB, max volume 8 TB (but practical 2 TB). No permissions, no journaling.
- **Advantage:** Simple, widely compatible.

NTFS (New Technology File System)

- **Windows default** since NT 3.1.
- **Features:** Journaling (\$LogFile), security (ACLs), compression, encryption (EFS), file-size limit 16 EB, hard links, reparse points (symlinks, mount points).
- **Structure:** Master File Table (MFT) – each file has one or more MFT records; small files stored directly in MFT (“resident”).
- **Performance:** Good for large volumes; supports disk quotas.

ext4 (Fourth Extended File System)

- **Linux default** for many distributions.
- **Features:** Journaling, extents (replaces block mapping for large files), delayed allocation, 1 EB max volume, subdirectory limits up to 64,000 (or higher with dir_index).
- **Backward compatibility:** Can mount ext2/ext3.
- **Tools:** `mkfs.ext4`, `tune2fs`.

Comparison table:

Feature	FAT32	NTFS	ext4
Max file size	4 GB	16 EB	16 TB (theoretically 1 EB)
Max volume size	2 TB (8 TB with 64KB clusters)	16 EB	1 EB
Journaling	No	Yes (metadata)	Yes (metadata, optional data)
Permissions	No	ACLs (yes)	POSIX (yes)
Compression	No	Yes	No (can use ecryptfs)
Default OS	Legacy Windows, USB drives	Windows	Linux

Summary

Concept	Key takeaway
File attributes	Name, ID, size, protection, timestamps, owner.
File operations	create, open, read, write, close, delete, truncate.
Directory structures	Single-level, two-level, tree, acyclic graph (with links), general graph.
Mounting	Attaching a file system to a directory path.
Contiguous allocation	Simple but causes fragmentation.
Linked allocation (FAT)	Good for growth, poor random access.
Indexed allocation (inode)	Direct + indirect pointers; supports large files.
Free space management	Bitmap, linked list, grouping, counting.
Disk scheduling	FCFS, SSTF, SCAN, C-SCAN, LOOK, C-LOOK.
File system layout	Boot block, superblock, inode table, data blocks.
Journaling	Write log before changes; recover after crash.
Examples	FAT32 (simple, universal), NTFS (feature-rich, Windows), ext4 (Linux standard).

File systems are the bridge between the OS and persistent storage. The next chapter covers I/O systems and device management.

Chapter 9: Input/Output Systems

Input/Output (I/O) is how the computer communicates with the outside world. This chapter covers the hardware and software that make I/O possible, from device controllers to interrupt handling, DMA, and disk arrays.

I/O Hardware: Ports, Buses, Controllers

I/O devices connect to the computer through a hierarchy of hardware components.

Ports

A **port** is a physical connector on the computer (e.g., USB port, Ethernet port, HDMI). It provides electrical and logical interface for a device. Each port is associated with one or more I/O addresses.

Buses

A **bus** is a set of wires shared by multiple devices. Common buses:

Bus	Typical use	Speed
PCIe	Graphics cards, NVMe SSDs	Several GB/s
SATA	Hard disks, SSDs	6 Gb/s
USB	Keyboards, mice, flash drives	Up to 40 Gb/s (USB4)
Memory bus	CPU ↔ RAM	Very high (50-100 GB/s)

The **system bus** connects CPU, memory, and I/O bridges. The **I/O bus** (e.g., PCIe) connects I/O controllers.

Controllers (Adapters)

A **controller** is a chip or circuit that manages a specific type of device. It handles low-level protocols, buffering, and error correction. The OS communicates with the controller via registers (control, status, data).

Example: A disk controller receives read/write commands, translates logical block addresses to physical locations, and transfers data.

Real-life analogy: A shipping port. The ship (device) docks at a specific berth (port). Cranes and customs officers (controller) unload and inspect cargo. The road network (bus) transports goods to the city.

Polling vs Interrupts

The CPU needs to know when an I/O device is ready for data or has completed an operation. Two basic strategies.

Polling

The CPU continuously checks the device's status register in a loop. When the status indicates "ready," the CPU performs the I/O.

```
while ((status_register & READY_BIT) == 0) {  
    // wait (busy loop)  
}  
// device ready – perform I/O
```

- **Pros:** Simple, no hardware overhead.
- **Cons:** Wastes CPU cycles (busy waiting), especially if devices are slow.

Interrupts

The device sends a hardware **interrupt** signal to the CPU when it needs attention. The CPU finishes the current instruction, saves state, and jumps to the **interrupt handler** (interrupt service routine, ISR). After handling, it returns to the interrupted process.

- **Pros:** CPU is free to do other work while device is busy.
- **Cons:** Overhead of saving/restoring state; interrupt storms can overwhelm the system.

Real-life analogy:

- **Polling:** A chef constantly looking at the oven every second to see if the cake is ready.
- **Interrupt:** The oven beeps (interrupt) when the cake is done; the chef continues chopping vegetables until then.

Trade-off: Use polling for very fast devices (e.g., network card with always-ready data) to avoid interrupt overhead. Use interrupts for slow devices (disks, keyboards).

Direct Memory Access (DMA)

Without DMA, the CPU must copy every byte from the device to memory (Programmed I/O – PIO). This ties up the CPU for large transfers.

DMA controller is special hardware that transfers data directly between device and memory without CPU involvement. The CPU sets up the transfer (source, destination, count), then continues other work. When the transfer completes, the DMA controller sends an interrupt.

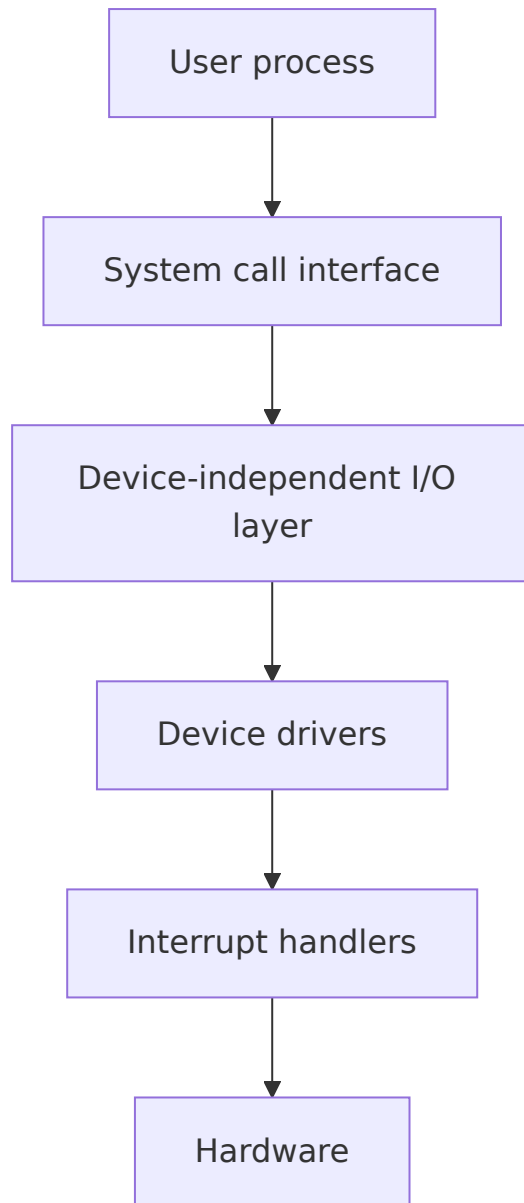
Steps:

1. CPU programs DMA controller: memory address, device, transfer size, direction.
2. DMA controller moves data (word by word or block by block) over the bus.
3. DMA controller asserts interrupt when done.

Real-life analogy: A moving company. Without DMA (PIO), you carry each box yourself (CPU-bound). With DMA, you hire movers – you tell them “move everything from room A to room B” and they do it while you do something else; they ring the bell when done.

I/O Software Layers

Modern operating systems organise I/O into hierarchical layers, hiding complexity and providing uniform interfaces.



1. Interrupt Handlers

Lowest layer. When an interrupt occurs, the handler:

- Saves registers.
- Acknowledges the interrupt.
- Schedules a callback or wakes a waiting process.
- Restores context and returns.

2. Device Drivers

Driver is a **kernel module** that knows how to talk to a specific device (or class of devices). It translates generic OS commands (e.g., `read block 123`) into device-specific register writes.

- Each driver implements a standard interface (open, read, write, ioctl).
- Approximately 70% of OS code can be in drivers (Linux, Windows).

3. Device-Independent I/O Layer

Provides common services across all devices:

- **Uniform naming:** /dev/sda , C:\ .
- **Protection:** Who can access which device.
- **Buffering:** Temporary storage to smooth data flow.
- **Error handling:** Retry, report, or ignore failures.
- **Allocation:** Manage shared devices (e.g., tape drives).

4. User-Space I/O

Library functions (e.g., printf , fread) that call system calls (read , write). Some I/O can be done entirely in user space (e.g., memory-mapped files, user-mode drivers in microkernels).

Real-life analogy:

- **Interrupt handler:** Fire alarm bell – immediate action.
- **Driver:** The building's electrician who knows exactly which wire controls which light.
- **Device-independent layer:** Building codes and safety standards that apply to all electrical work.
- **User-space:** You flipping a light switch.

Buffering, Caching, Spooling

These techniques improve I/O performance and handle speed mismatches.

Buffering

A **buffer** is a memory area that temporarily holds data while it is transferred between two entities (e.g., between device and application). Buffers decouple producer and consumer speeds.

Types:

- **Single buffer:** OS reads data into kernel buffer, then copies to user space.

- **Double buffering:** Use two buffers – one is filled while the other is being processed.
- **Circular buffer:** Multiple buffers in a ring.

Real-life: A bread factory (fast producer) and a toaster (slow consumer). A delivery table (buffer) collects loaves; the toaster picks them up when ready.

Caching

A **cache** holds recently used data in faster storage to reduce repeated slower I/O. Caching is different from buffering: cache stores **already used** data for possible reuse; buffer stores **in-transit** data.

Example: Page cache in Linux – recently read disk blocks kept in free memory.

Spooling (Simultaneous Peripheral Operation On-Line)

A **spool** is a buffer that holds output for a slow device (e.g., printer). Each process writes its output to a spool file on disk. A background daemon (printer daemon) later prints the files.

Advantage: Processes do not wait for the printer; they continue immediately.

Real-life: A receptionist's inbox – people drop messages (spool) and the receptionist delivers them one by one.

Block and Character Devices

Type	Access	Examples	Typical interface
Block device	Fixed-size blocks, random access	Disk, SSD, USB drive	<code>read()</code> , <code>write()</code> , <code>seek()</code>
Character device	Stream of bytes, no seek	Keyboard, mouse, serial port	<code>getchar()</code> , <code>putchar()</code> (no random access)

In Unix, block devices appear as files in `/dev` . Character devices also appear (e.g., `/dev/tty`). Some devices (e.g., network interfaces) are neither – they use socket interface.

Disk Structure

Magnetic hard disks (HDDs) are still widely used; understanding their geometry helps optimise file systems.

Components

A disk consists of:

- **Platters:** Circular metal/glass disks coated with magnetic material.
- **Spindle:** Rotates platters (5400, 7200, 10000 RPM).
- **Read/write heads:** One per platter surface, mounted on an **arm**.
- **Actuator:** Moves the arm across the platter.

Geometric terms

Term	Meaning
Track	Concentric circle on a platter surface.
Cylinder	Set of tracks at the same arm position across all platters.
Sector	Smallest addressable unit (traditionally 512 bytes, now 4 KB).
Head	The read/write mechanism for one surface.

Capacity = (bytes per sector) × (sectors per track) × (tracks per cylinder) × (cylinders) × (platters).

Real-life analogy: A vinyl record player. The record (platter) spins. The needle (head) moves radially to different grooves (tracks). The same groove position across multiple records stacked vertically is a cylinder.

RAID Levels

RAID (Redundant Array of Independent Disks) combines multiple physical disks into one logical unit for performance, reliability, or both.

RAID 0 (Striping)

Data is striped across disks in blocks. No redundancy.

- **Pros:** High performance (parallel I/O).

- **Cons:** Any disk failure loses all data.
- **Use:** Temporary data, high-performance scratch space.

RAID 1 (Mirroring)

Every write goes to two (or more) disks. Reads can be from either.

- **Pros:** Excellent fault tolerance; good read performance.
- **Cons:** $2 \times$ storage cost.
- **Use:** OS disks, critical databases.

RAID 5 (Striping with Parity)

Blocks and parity are striped across all disks. Parity is distributed (not on a single disk). Can tolerate **one** disk failure.

- **Pros:** Good read performance; efficient storage ($1/N$ overhead).
- **Cons:** Slow writes (read-modify-write penalty).
- **Use:** File servers, general-purpose storage.

RAID 6

Like RAID 5 but with two parity blocks (different algorithms). Tolerates **two** disk failures.

- **Pros:** Very high fault tolerance.
- **Cons:** Even slower writes; $2/N$ overhead.
- **Use:** Large capacity systems, archival storage.

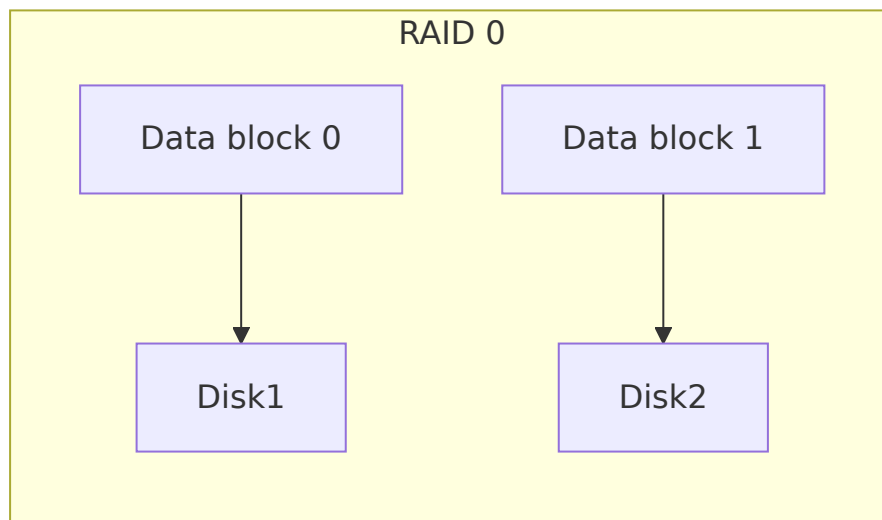
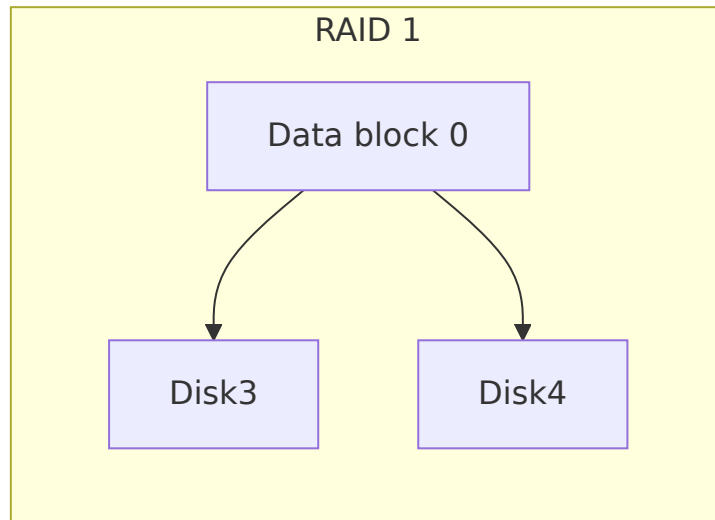
RAID 10 (1+0)

Striped mirrors: first mirror, then stripe across mirror pairs (or stripe then mirror – both exist). Combines RAID 0 performance with RAID 1 fault tolerance.

- **Pros:** Excellent performance and fault tolerance (can lose one disk per mirror pair).
- **Cons:** Expensive ($2 \times$ storage).
- **Use:** High-performance databases, virtualisation hosts.

RAID	Min disks	Fault tolerance	Read perf	Write perf	Capacity
0	2	None	High	High	$N \times \text{disk}$
1	2	1 disk	High	Medium	$N/2 \times \text{disk}$

5	3	1 disk	High	Low	$(N-1) \times \text{disk}$
6	4	2 disks	High	Very low	$(N-2) \times \text{disk}$
10	4	Up to 2 (one per pair)	Very high	High	$N/2 \times \text{disk}$



Real-life analogy:

- **RAID 0:** Two runners in a relay holding half the message each – fast, but if one runner falls, the whole message is lost.
 - **RAID 1:** Two students writing the same notes – if one loses their notebook, the other still has the complete information.
 - **RAID 5:** A group of 4 friends who calculate a checksum for their shared document. If one friend leaves, the others can reconstruct missing parts from the checksum.
-

Summary

Concept	Key takeaway
I/O hardware	Ports (connectors), buses (communication paths), controllers (device managers).
Polling	CPU continuously checks device status – simple but wastes cycles.
Interrupts	Device signals CPU when ready – efficient for slow devices.
DMA	Hardware transfers data without CPU; only setup and completion interrupt.
I/O software layers	Interrupt handlers → device drivers → device-independent I/O → user-space I/O.
Buffering	Temporary storage for in-transit data (smooths speed mismatches).
Caching	Store reused data for faster future access.
Spooling	Buffer output for slow devices (printers).
Block vs character	Block devices: random access, fixed blocks; character: byte stream.
Disk structure	Tracks, cylinders, sectors – geometry affects scheduling.
RAID levels	0 (striping, no redundancy), 1 (mirroring), 5 (parity, one fault), 6 (two faults), 10 (mirrored stripes).

I/O systems complete the OS puzzle, bridging internal abstractions to the physical world. This chapter concludes our journey through operating systems fundamentals.

Chapter 10: Security and Protection

Security and protection are fundamental to any operating system. This chapter covers the goals of security, authentication methods, access control models, protection mechanisms, common attacks, and encryption techniques that safeguard data and resources.

Security Goals: Confidentiality, Integrity, Availability

The **CIA triad** defines the core objectives of information security.

Goal	Meaning	Example failure
Confidentiality	Preventing unauthorised access to information.	Hacker steals password file.
Integrity	Ensuring data is not tampered with or modified without permission.	Malware alters system binaries.
Availability	Ensuring systems and data are accessible when needed.	Denial-of-service (DoS) attack makes website unreachable.

Real-life analogy: A bank vault:

- **Confidentiality:** Only authorised staff can see inside.
- **Integrity:** Money stored cannot be altered (counterfeiting prevented).
- **Availability:** The vault is accessible during business hours.

Authentication

Authentication verifies the identity of a user or process. Common factors:

1. Something you know (passwords, PINs)

Most common, but vulnerable to guessing, theft, phishing.

- **Password storage:** Modern OSes store **hashed** passwords (not plaintext) using algorithms like bcrypt, SHA-256 with salt.
- **Salt:** Random value added to each password before hashing to prevent rainbow table attacks.

2. Something you have (smart card, security token, phone)

Adds a second factor; usually combined with password (two-factor authentication – 2FA).

- **Hardware tokens:** YubiKey, RSA SecurID.
- **Software tokens:** TOTP (Google Authenticator) – time-based one-time password.

3. Something you are (biometrics)

Fingerprint, iris scan, face recognition, voice.

- **Pros:** Hard to forge, convenient.
- **Cons:** Not changeable if compromised; false positives/negatives.

Multi-factor Authentication (MFA)

Combines two or more factors. For example, password (knowledge) + SMS code (possession). Greatly increases security.

Real-life analogy: Entering a secure building:

- Password = keycode at the door.
- Security token = keycard.
- Biometric = fingerprint scanner.
- MFA = keycard + PIN.

Access Control

Once authenticated, the OS must decide what resources a user can access and what operations are allowed. Three classic models.

DAC (Discretionary Access Control)

The **owner** of a resource decides who can access it. Common in general-purpose OSes (Unix permissions, Windows ACLs).

- **Unix permissions:** Read, write, execute for owner, group, others.

- **Problem:** Users may accidentally grant excessive permissions.

Real-life: You own a house; you give keys to friends as you wish.

MAC (Mandatory Access Control)

The **system** (OS) enforces a global policy that users cannot override. Used in high-security environments (military, SELinux, AppArmor).

- **Labels:** Each resource and user has a security label (e.g., “Top Secret” , “Secret” , “Unclassified”).
- **Rules:** A user can access a resource only if their clearance dominates the resource’s label.

Real-life: Military document classification – a “Secret” clearance holder cannot read “Top Secret” documents, regardless of their wishes.

RBAC (Role-Based Access Control)

Permissions are assigned to **roles**, and users are assigned to roles. Roles reflect job functions (e.g., “Doctor” , “Nurse” , “Accountant”).

- **Advantage:** Simplify administration – change role permissions, not per user.
- **Examples:** Database systems, enterprise applications, Windows groups.

Real-life: Hospital – doctors have prescription rights; nurses have patient care rights; receptionists can only schedule appointments. Users are assigned to roles.

Protection Mechanisms: Capabilities and Access Matrix

The OS needs internal structures to enforce access control.

Access Matrix

A conceptual table where rows represent **subjects** (users, processes) and columns represent **objects** (files, devices). Each cell contains the allowed **operations** (read, write, execute, delete).

Subject	File A	File B	Printer
---------	--------	--------	---------

Alice	r,w	r	(none)
Bob	r	r,w	print

- **Implementation:**

- **Access Control Lists (ACLs):** Attach a list to each object (column of matrix). Windows NTFS, POSIX ACLs.
- **Capabilities:** Attach a list to each subject (row of matrix). Each capability is an unforgeable token granting access to a specific object with specific rights.

Capability

A capability is a ticket that gives a process permission to perform certain operations on an object. Often implemented as a protected, kernel-managed reference.

- **Advantage:** Easy to delegate; fine-grained.
- **Disadvantage:** Need to revoke capabilities (can be complex).
- **Examples:** Early research OS (Hydra, KeyKOS), some microkernels (seL4).

Real-life analogy:

- **ACL:** A bouncer's list at a club – each club has its own list of allowed guests.
- **Capability:** A wristband – you show it to enter; different colours give different access.

Principle of Least Privilege

The **principle of least privilege** states that a process (or user) should be granted only the minimum privileges necessary to perform its task, and only for the minimum time needed.

- **Benefit:** Limits damage from bugs or attacks.
- **Examples:**
 - Unix `setuid` programs drop privileges after initial setup.
 - Web browsers run tabs in sandboxes with limited file access.
 - Linux capabilities: break root privileges into small pieces (`CAP_NET_ADMIN` , `CAP_SYS_ADMIN`).

Real-life: A hotel valet – they can park your car but cannot open your room safe.

Malicious Software (Malware)

Malware is software designed to harm or infiltrate a system.

Type	Description	Propagation	Payload
Virus	Attaches to legitimate programs; requires user action (running infected program).	File infection	Corruption, theft, backdoor.
Worm	Self-replicates across networks without user action.	Network exploits, emails	Resource exhaustion, backdoor.
Trojan	Disguised as legitimate software; does not self-replicate.	Social engineering	Data theft, ransomware.
Rootkit	Hides its presence (files, processes, network connections) from OS.	Often installed after initial compromise	Stealth, persistent backdoor.
Ransomware	Encrypts user files and demands payment.	Trojan, worm, exploit	File encryption.
Spyware	Collects user information secretly.	Bundled with freeware	Keylogging, browsing data.

Real-life analogy:

- **Virus:** A contagious disease spread by touch.
- **Worm:** A airborne pathogen that spreads itself through ventilation systems.
- **Trojan:** A delivery box labeled “gift” that contains a bomb.
- **Rootkit:** A spy who can delete security camera footage so you never know they were there.

Buffer Overflow Attacks and Defenses

A **buffer overflow** occurs when a program writes data beyond the allocated buffer bounds, overwriting adjacent memory. Attackers exploit this to inject malicious code or change

execution flow.

Classic stack overflow

```
void vulnerable(char *str) {
    char buffer[64];
    strcpy(buffer, str);    // no bounds check
}
```

If `str` is longer than 64 bytes, it overwrites the return address on the stack. An attacker crafts input that places attacker code (shellcode) and sets the return address to point to it.

Defenses

Defense	Description	Effectiveness
Non-executable stack (NX bit)	Mark stack memory as non-executable. Prevents shellcode execution.	Bypassed by return-to-libc or ROP.
Address Space Layout Randomization (ASLR)	Randomise base addresses of stack, heap, libraries. Attacker cannot know return address location.	Effective; brute-force possible on 32-bit.
Stack canaries	Place a secret value (canary) before return address. If overflow overwrites it, program aborts.	Very effective; used by GCC (<code>-fstack-protector</code>).
Bounds checking	Compile-time or runtime checks (e.g., Rust, Safe C extensions).	Most thorough, but performance cost.
Control Flow Integrity (CFI)	Restrict program jumps to a precomputed set of valid targets.	Emerging standard (e.g., Intel CET).

Real-life analogy: A hotel keycard system. Buffers are like mail slots – if you shove in a long package, it might push the room door open. Defenses are like making the slot too narrow (little canary) or randomising which door the key opens (ASLR).

Encryption Basics for OS Security

Encryption transforms data to prevent unauthorised reading. Modern OSes use encryption to protect data at rest (disk) and in transit (network).

Symmetric Encryption

Same key encrypts and decrypts.

- **Algorithms:** AES (Advanced Encryption Standard), ChaCha20.
- **Use:** Full-disk encryption (FDE), file encryption, network session encryption (TLS).

Asymmetric Encryption (Public Key)

Two keys: public key (encrypts), private key (decrypts). Infeasible to derive private from public.

- **Algorithms:** RSA, ECC (Elliptic Curve Cryptography).
- **Use:** Key exchange, digital signatures, authentication.

Full-Disk Encryption (FDE)

Encrypts the entire storage device (except boot block). The OS decrypts on-the-fly using a key derived from a passphrase, TPM (Trusted Platform Module), or USB token.

- **Examples:** Windows BitLocker, macOS FileVault, Linux LUKS (dm-crypt).
- **Threat model:** Protects against physical theft of device; does not protect against a compromised running system.

How LUKS works (simplified):

1. User enters passphrase.
2. Passphrase decrypts a master key stored in LUKS header.
3. Master key decrypts disk blocks on-the-fly.

Real-life analogy: A safe (encrypted disk) with a combination lock (passphrase). Even if a thief steals the safe, they cannot read documents inside without the combination.

Summary

Concept	Key takeaway
Security goals	Confidentiality, integrity, availability (CIA triad).

Authentication	Something you know (password), have (token), are (biometric); MFA combines factors.
Access control	DAC (owner decides), MAC (system decides), RBAC (roles).
Protection mechanisms	Access matrix; implemented as ACLs (per object) or capabilities (per subject).
Least privilege	Grant minimal rights for minimal time to limit damage.
Malware types	Viruses (attach), worms (self-spread), trojans (disguised), rootkits (hide), ransomware (encrypt).
Buffer overflow defenses	NX bit, ASLR, stack canaries, bounds checking, CFI.
Encryption for OS	Full-disk encryption (BitLocker, FileVault, LUKS) protects against physical theft.

Security is not a product but a continuous process. Understanding these OS-level mechanisms is the first step toward building and maintaining trustworthy systems.

Chapter 11: Virtualization and Cloud Computing

Virtualization decouples software from the underlying hardware, enabling multiple operating systems to run on a single physical machine. Cloud computing builds on virtualization to deliver computing resources as on-demand services. This chapter explains hypervisors, containers, cloud models, and their trade-offs.

Virtual Machine Concepts: Hypervisors

A **virtual machine (VM)** is a software implementation of a physical computer that runs an operating system and applications. The **hypervisor** (or virtual machine monitor, VMM) is the software layer that creates and manages VMs.

Type 1 Hypervisor (Bare-Metal)

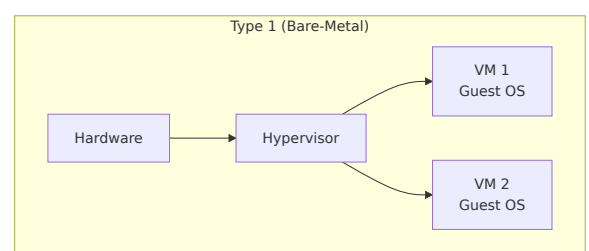
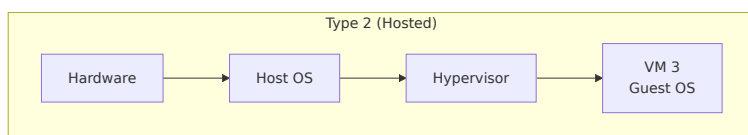
Runs directly on the physical hardware without a host OS. It has direct access to hardware resources.

- **Examples:** VMware ESXi, Microsoft Hyper-V, KVM (Linux kernel-based), Xen.
- **Pros:** High performance, low overhead, better security.
- **Cons:** Requires hardware drivers for all devices; more complex management.

Type 2 Hypervisor (Hosted)

Runs as an application on top of a host operating system. The host OS manages physical hardware.

- **Examples:** Oracle VirtualBox, VMware Workstation, QEMU (without KVM acceleration).
- **Pros:** Easier to install, good for development and testing.
- **Cons:** Higher overhead (calls go through host OS), lower performance.



Real-life analogy:

- **Type 1:** A high-rise apartment building with a property manager living on-site (hypervisor directly controls elevators, electricity).
 - **Type 2:** A rental agency that runs inside a larger office building – you rent a room, but the building's own facilities manager (host OS) controls the infrastructure.
-

Virtualization vs Emulation

	Virtualization	Emulation
Guest instructions	Run directly on CPU (if same architecture)	Simulated in software
Performance	Near-native (with hardware support)	Very slow (10-100× slowdown)
Guest OS modification	Not required (full virtualization) or minimal (para-virtualization)	None needed
Use case	Run x86 Linux on x86 Windows	Run ARM software on x86, retro-gaming

Virtualization requires the guest and host to have the same instruction set architecture (ISA). **Emulation** can run software for any architecture by interpreting or translating each instruction.

Example: QEMU can emulate an ARM CPU on an x86 host (emulation). QEMU + KVM can virtualize an x86 guest on an x86 host (virtualization).

Real-life analogy:

- **Virtualization:** A movie theatre where films are shown in their native format (digital projector plays digital files).
 - **Emulation:** A translator who reads a French script out loud in English – every word is processed.
-

Para-Virtualization and Hardware-Assisted Virtualization

Para-Virtualization

The guest OS is modified to be aware of the hypervisor. Instead of trapping privileged instructions (which are slow to emulate), the guest makes explicit **hypercalls** to the hypervisor.

- **Pros:** High performance (reduces trapping overhead).
- **Cons:** Requires modifying the guest OS (not possible for Windows).
- **Example:** Xen's early approach (paravirtualized Linux, NetBSD).

Hardware-Assisted Virtualization

Modern CPUs (Intel VT-x, AMD-V) add hardware support for virtualization:

- **New CPU modes:** VMX root (hypervisor) and VMX non-root (guest).
- **Extended Page Tables (EPT) / Nested Page Tables (NPT):** Hardware-assisted virtual memory translation (reduces shadow page table overhead).
- **VMCS (Virtual Machine Control Structure):** Stores guest state, exit reasons.

Benefits:

- Full virtualization without guest modification.
- Near-native performance.
- Simpler hypervisor implementation.

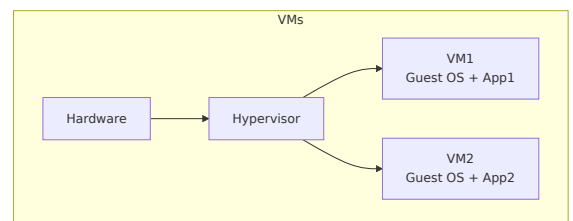
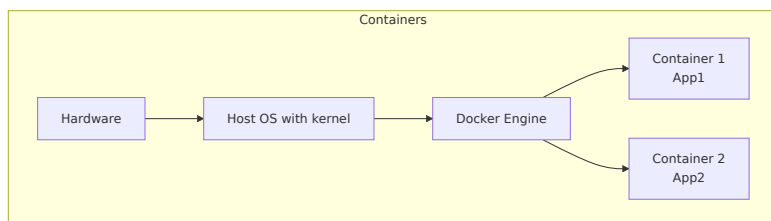
Real-life analogy:

- **Para-virtualization:** A foreign tourist who learns a few local customs (modified OS) to move faster through airport security.
- **Hardware-assisted:** An express lane at immigration with automated passport scanners (CPU support) – everyone can use it without special training.

Containers vs Virtual Machines

Containers (e.g., Docker, LXC, Podman) virtualize at the **OS level**, not hardware level. They share the host kernel but isolate processes in namespaces.

Feature	Virtual Machine	Container
Isolation	Strong (separate kernels)	Moderate (shared kernel, namespaced)
Startup time	Seconds to minutes (boot OS)	Milliseconds (just start process)
Disk footprint	GBs (full OS + apps)	MBs (app + dependencies)
Performance overhead	Small (type 1) or moderate (type 2)	Very low (near-native)
Kernel	Each VM has its own kernel	All containers share host kernel
Guest OS	Any OS (Linux, Windows, BSD)	Must match host kernel (e.g., Linux host → Linux containers; Windows host → Windows containers)



Container primitives (Linux):

- **Namespaces:** Isolate views (PID, network, mount, UTS, IPC, user).
- **Cgroups (control groups):** Limit resource usage (CPU, memory, disk I/O).
- **Union filesystems** (OverlayFS, AUFS): Efficient, layered images.

Real-life analogy:

- **Virtual machines:** Detached houses – each has its own plumbing, electricity, foundation. Very isolated but expensive and slow to build.
- **Containers:** Apartments in the same building – share central heating, water, and elevator (host kernel), but have separate doors (namespaces) and rent control (cgroups).

Cloud Computing Models

Cloud computing delivers computing resources over the internet on a pay-per-use basis.
Three main service models.

IaaS (Infrastructure as a Service)

Provides virtual machines, storage, networks, and other fundamental computing resources.
You control the OS, applications, and middleware.

- **Examples:** AWS EC2, Google Compute Engine, Microsoft Azure VMs.
- **Customer responsibility:** OS patches, security, application data.
- **Analogy:** Renting a plot of land – you build whatever you want.

PaaS (Platform as a Service)

Provides a platform where you deploy applications without managing the underlying infrastructure (OS, runtime, middleware, databases).

- **Examples:** AWS Elastic Beanstalk, Google App Engine, Heroku.
- **Customer responsibility:** Application code and data.
- **Analogy:** Renting a fully equipped kitchen – you only bring the ingredients and cook.

SaaS (Software as a Service)

Provides ready-to-use applications over the internet. No installation, no management of infrastructure or platform.

- **Examples:** Google Workspace (Gmail, Docs), Salesforce, Microsoft 365.
- **Customer responsibility:** Only user data and configuration.
- **Analogy:** Eating at a restaurant – you just order and consume.

Deployment models:

- **Public cloud:** Shared infrastructure over the internet.
- **Private cloud:** Dedicated infrastructure for one organisation.
- **Hybrid cloud:** Combined public and private clouds.
- **Multi-cloud:** Using multiple public cloud providers.

Challenges: Isolation, Performance, Security

Isolation

VMs provide strong isolation (separate kernels). However, side-channel attacks (e.g., Spectre, Meltdown) can leak information across VMs on the same physical host.

Containers share the host kernel. A kernel bug or container escape can compromise other containers. Mitigations: seccomp, SELinux, AppArmor, user namespaces, gVisor (sandboxed container runtime).

Real-life: Apartment walls (containers) provide less sound isolation than separate houses (VMs). A loud party in one apartment disturbs neighbours.

Performance

Overhead	VMs	Containers
CPU (native)	~0-5% (with hardware assist)	~0-1%
Memory	Fixed per VM (static)	Dynamic, copy-on-write
I/O (disk, network)	Higher (double-buffering)	Near-native
Boot time	Long (OS boot)	Instant (process start)

Considerations:

- **VMs:** Better for mixed-trust workloads, legacy apps, different OSes.
- **Containers:** Better for microservices, CI/CD, high density.

Security

Shared risks (both VMs and containers):

- Hypervisor or kernel vulnerabilities.
- Misconfigured networking (exposed ports).
- Insecure images or base OS.

VM-specific:

- Hypervisor attacks (VM escape).
- Resource exhaustion (fork bombs affecting others).

Container-specific:

- Privilege escalation from container to host.
- Secret leakage via environment variables.

- Image supply chain attacks (malicious layers).

Best practices:

- Keep hypervisor/host kernel patched.
- Use minimal base images (Alpine, Distroless).
- Run containers as non-root, with read-only root filesystems.
- Use secure secrets management (HashiCorp Vault, Kubernetes secrets).
- Network policies for micro-segmentation.

Summary

Concept	Key takeaway
Hypervisor Type 1	Bare-metal, runs directly on hardware (ESXi, KVM, Hyper-V).
Hypervisor Type 2	Runs as app on host OS (VirtualBox, Workstation).
Virtualization vs emulation	Virtualization: same ISA, near-native. Emulation: different ISA, slow.
Para-virtualization	Guest modified to use hypercalls; more efficient (Xen).
Hardware-assisted virt	CPU extensions (VT-x, AMD-V) simplify hypervisors and improve performance.
Containers	OS-level virtualization, share host kernel, lightweight (Docker, LXC).
VMs vs containers	VMs: strong isolation, slower; containers: fast, dense, shared kernel risk.
Cloud IaaS	Infrastructure – VMs, storage, networks (AWS EC2).
Cloud PaaS	Platform – deploy apps without managing OS (Heroku, App Engine).
Cloud SaaS	Software – ready-to-use apps (Gmail, Office 365).
Challenges	Isolation (side-channel, container escape), performance trade-offs, security hardening.

Virtualization and cloud computing have transformed how we build and deploy software. Understanding these concepts bridges traditional OS knowledge with modern DevOps practices.

Chapter 12: Distributed Operating Systems (Overview)

A distributed operating system manages a collection of independent computers (nodes) that appear to users as a single coherent system. This chapter introduces the core concepts: system models, communication, synchronization, mutual exclusion, distributed file systems, and consensus algorithms.

Distributed System Models

Distributed systems can be classified by how nodes interact and share resources.

Client-Server Model

One or more **servers** provide services (e.g., file storage, database) to **clients** that request them. Servers are passive (wait for requests); clients are active (initiate requests).

- **Examples:** Web browsing (browser client, web server), email (mail client, mail server).
- **Communication:** Request-reply over network (often TCP/IP).



Peer-to-Peer (P2P) Model

All nodes (peers) are equal – each can act as both client and server. No central coordinator. Resources are shared directly between peers.

- **Examples:** BitTorrent (file sharing), Bitcoin (blockchain), Skype (early version).
- **Challenges:** Discovery, security, fault tolerance, incentive mechanisms.

Real-life analogy:

- **Client-server:** A library with a central librarian (server) – patrons (clients) ask the librarian for books.
- **Peer-to-peer:** A study group where each member brings books and shares directly – no leader.

Remote Procedure Call (RPC) and Message Passing

Distributed components communicate by sending messages over a network. Two dominant models.

Message Passing

Processes explicitly send and receive messages using `send(destination, message)` and `receive(source, message)`.

- **Synchronous (blocking):** Sender waits until receiver accepts the message. Simpler but slower.
- **Asynchronous (non-blocking):** Sender continues immediately; messages are buffered. More efficient but requires care.

Remote Procedure Call (RPC)

Makes communication look like a local procedure call. The client calls a function; the RPC system marshals arguments, sends a request to the server, unmarshals results, and returns them.

Steps:

1. Client calls `foo(args)` (client stub).
2. Stub marshals args into a message.
3. Transport sends message to server.
4. Server stub unmarshals args, calls local `foo`.
5. Results marshalled and sent back.
6. Client stub unmarshals result and returns to client.

Challenges:

- **Binding:** How does client know server's address? (Registry, naming service)
- **Failure modes:** Server crash, network partition, lost messages.
- **Idempotency:** May need to retry without causing duplicate effects.

Real-life analogy: A phone call with an interpreter. You speak in English (local call); interpreter translates to Japanese (message), the other person answers; interpreter translates back. You don't need to know the language or the phone network details.

Network Transparency and Naming

Network transparency means that users and applications cannot tell whether a resource is local or remote. To achieve this, distributed systems need a unified **naming** scheme.

Naming Requirements

- **Location transparency:** The name of a resource does not reveal its physical location (e.g., `file://server/share/doc.txt` – not fully transparent; better: UUID).
- **Location independence:** The resource can move without changing its name.

Approaches

Approach	Example	Transparency
Host+path	NFS: <code>server:/path</code>	Low (name includes server)
Global namespace	AFS (Andrew File System) – <code>/afs/cs.cmu.edu/file</code>	Medium (cell name included)
Unique identifiers	UUIDs, content-addressed storage (IPFS)	High (location opaque)

Real-life analogy:

- **Host+path:** “John in Room 204” (not transparent – move John, name changes).
- **Global namespace:** “John, employee ID 4711” (transparent – HR knows where John sits).
- **UUID:** John’s biometric template – you scan his fingerprint to find him anywhere.

Distributed Synchronization: Lamport Clocks and Vector Clocks

In distributed systems, physical clocks on different nodes may drift. To order events, we use **logical clocks**.

Lamport Clock (Lamport Timestamp)

Each process maintains a counter. Rules:

1. On an event inside a process, increment counter.
2. When sending a message, include current counter.
3. On receiving a message, set local counter = $\max(\text{local}, \text{received}) + 1$.

Happens-before relation ($\rightarrow$):

- If a and b are events in same process and a occurs before b, then $a \rightarrow b$.
- If a sends a message and b receives it, then $a \rightarrow b$.
- If $a \rightarrow b$ and $b \rightarrow c$, then $a \rightarrow c$.

Lamport clocks satisfy: if $a \rightarrow b$ then $L(a) < L(b)$. However, $L(a) < L(b)$ does not imply $a \rightarrow b$ (concurrent events may have arbitrary order).

Vector Clock

Each process keeps a vector of length N (number of processes). Rules:

- On internal event: increment own entry.
- On send: increment own entry, send entire vector.
- On receive: update each entry to $\max(\text{local}, \text{received})$, then increment own entry.

Properties:

- $V(a) < V(b)$ component-wise iff $a \rightarrow b$.
- Concurrent events have incomparable vectors.

Real-life analogy:

- **Lamport clock:** A counter in a bakery. Tickets tell the order of events but two tickets with 5 and 5 could be unrelated.
- **Vector clock:** A log where each person has a personal counter. Comparing logs reveals exactly who knew what when.

Distributed Mutual Exclusion

Multiple processes on different machines may need to access a shared resource (e.g., a printer, a file). Three classic algorithms.

1. Centralized Algorithm

One coordinator (server) grants permission. A process sends a request; if no one else holds the resource, coordinator grants it; otherwise, process is queued.

- **Pros:** Simple, fair, minimal messages (3 per critical section: request, grant, release).
- **Cons:** Single point of failure; coordinator bottleneck.

2. Distributed Algorithm (Ricart-Agrawala)

All processes are equal. When a process wants to enter a critical section, it sends a request (with Lamport timestamp) to **all** other processes. It enters only when all others reply “OK” .

- A process replies immediately if it does **not** want the resource, or if its own request has a **later** timestamp (higher number = lower priority). Otherwise, it queues the request.
- **Messages:** $2 \times (N-1)$ per entry (request + reply). High overhead.

3. Token-Based Algorithm (Token Ring)

A single token circulates among processes in a logical ring. A process may enter the critical section only when it holds the token.

- **Pros:** No starvation; low overhead when demand is low (token passes regardless).
- **Cons:** Lost token requires recovery; latency proportional to ring size.

Algorithm	Messages per CS	Fault tolerance	Coordinator
Centralized	3 (req, grant, rel)	Low (coordinator fails)	Yes
Ricart-Agrawala	$2(N-1)$	High (any process can recover)	No
Token ring	0 to N (token pass)	Medium (token loss)	No (logical ring)

Real-life analogy: A single-key restroom in an office:

- **Centralized:** One secretary (coordinator) hands out the key on request.
- **Distributed:** Every employee has a list of all others; you call everyone. If someone else is already in the restroom, they say “I’m busy” unless your timestamp (urgency) is higher.
- **Token-based:** Employees sit in a circle and pass a physical key around. Only the person holding the key can use the restroom.

Distributed File Systems (NFS, AFS)

Distributed file systems allow files on remote servers to be accessed as if they were local.

NFS (Network File System)

- **Developed by** Sun Microsystems (1984). Widely used in Unix/Linux.
- **Protocol:** Stateless – each request is independent (server does not remember client state).
- **Mounting:** Client mounts a remote directory to a local mount point (e.g., `mount server:/export /mnt`).
- **Transparency:** Partial – path includes server name or mount convention.
- **Cache consistency:** Weak (clients cache for few seconds; `close-to-open` consistency).

AFS (Andrew File System)

- **Developed by** Carnegie Mellon University (1980s).
- **Protocol:** Stateful – server tracks which clients have which files.
- **Whole-file caching:** When a client opens a file, the entire file is fetched to local disk (or memory). On close, if modified, whole file is sent back.
- **Scalability:** Uses cell hierarchy; excellent wide-area performance.
- **Consistency:** Callback mechanism – server promises to notify clients when a file changes.

Comparison

Feature	NFS	AFS
State	Stateless	Stateful (callbacks)
Caching	Block-level, short timeout	Whole-file, long-term
Scalability	Good for LAN	Excellent for WAN
Consistency	Close-to-open	Callback + version
Complexity	Simpler	More complex

Real-life analogy:

- **NFS:** A library where you can read pages one by one, but the librarian may re-shelve the book at any moment (stateless).
- **AFS:** You check out the entire book, take it home, and the library promises to call you if someone else needs it (callback).

Consensus: Paxos and Raft (Conceptual)

Consensus algorithms allow a set of distributed processes to agree on a single value even if some processes fail (crash or become unreliable). Used in distributed databases (e.g., etcd, ZooKeeper, Spanner).

Paxos

Developed by Leslie Lamport (1990s). Famous for being difficult to understand but robust.

Roles:

- **Proposer:** Suggests a value.
- **Acceptor:** Votes on proposals.
- **Learner:** Learns the chosen value.

Basic Paxos phases:

1. **Prepare:** Proposer sends `prepare(proposal_number)` to a majority of acceptors.
2. **Promise:** Acceptor promises never to accept lower-numbered proposals; returns last accepted value (if any).
3. **Accept:** Proposer sends `accept(proposal_number, value)` – value is either its own or the one returned.
4. **Accepted:** Acceptors accept if they haven't promised higher numbers; majority reached → consensus.

Paxos can be extended to **Multi-Paxos** for a sequence of decisions (log replication).

Raft

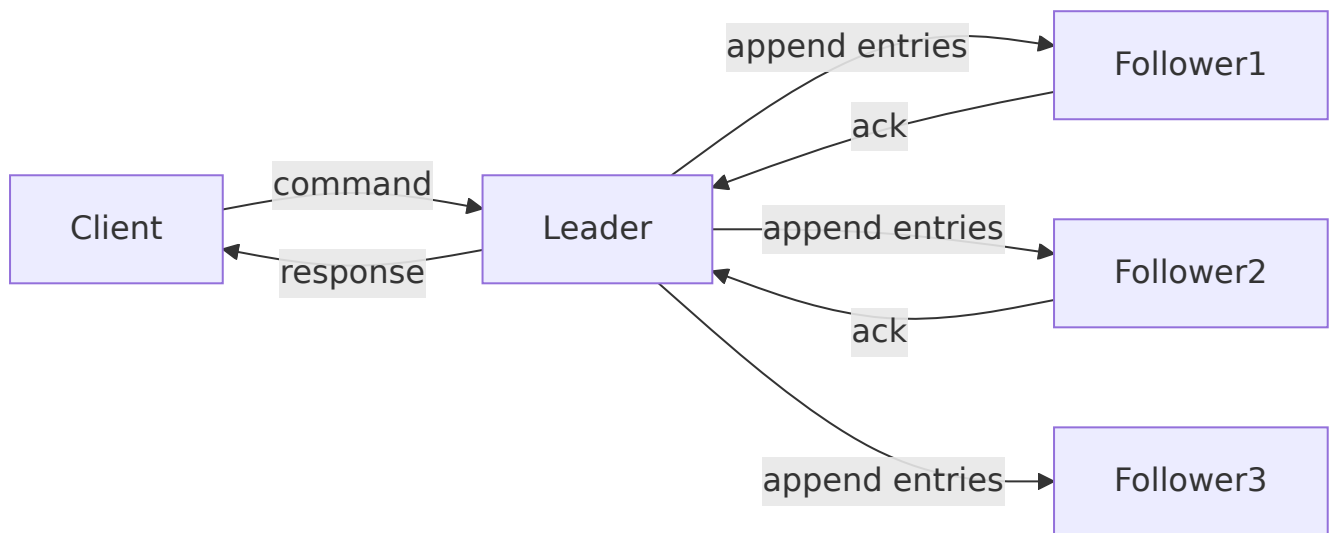
Designed to be more understandable than Paxos (Diego Ongaro, 2014). Uses **leader-based** approach.

Key components:

- **Leader election:** One server is leader; others are followers. If followers hear nothing from leader (timeout), they start an election.
- **Log replication:** Leader accepts client commands, appends to its log, replicates to followers. When a majority accept, the command is committed.
- **Safety:** Only the leader with the most up-to-date log can be elected (prevents overwritten entries).

Real-life analogy:

- **Paxos:** A committee without a chair. Each member can propose a motion; they vote in rounds, and some members may be absent (fail). Eventually, they agree on a single outcome – but the process is complicated.
- **Raft:** A team with an elected captain (leader). The captain tells everyone what to do. If the captain disappears, the team elects a new captain. Much simpler to follow.



Summary

Concept	Key takeaway
Client-server	Centralised server serves many clients (web, email).
Peer-to-peer	All nodes equal; no central coordinator (BitTorrent).
RPC	Remote call looks like local function call; hides network details.
Message passing	Explicit send/receive; more flexible but programmer-visible.
Network transparency	Resource names hide location; global namespaces or UUIDs.
Lamport clock	Logical clock orders events; if $a \rightarrow b$ then $L(a) < L(b)$.
Vector clock	Captures causal relationships; can detect concurrent events.

Distributed mutual exclusion	Centralised, distributed (Ricart-Agrawala), token-based algorithms.
NFS	Stateless, block-caching, simple, for LAN.
AFS	Stateful, whole-file caching, callbacks, for WAN.
Paxos	Classic consensus; robust but complex.
Raft	Leader-based consensus; designed for understandability.

Distributed operating systems extend the OS abstraction across multiple machines. While this chapter provided an overview, deeper study of networking, fault tolerance, and distributed algorithms is essential for building modern large-scale systems.

Chapter 13: Advanced Topics

This chapter covers specialised areas of operating systems that extend beyond general-purpose computing: real-time and embedded systems, mobile platforms, performance profiling, and fault tolerance. These topics highlight how OS principles adapt to different constraints and environments.

Real-Time Operating Systems (RTOS)

A **Real-Time Operating System** guarantees that tasks meet their timing deadlines. Correctness depends not only on logical results but also on when results are produced.

Hard vs Soft Real-Time

Type	Deadline violation consequence	Examples
Hard real-time	Catastrophic (system failure, injury)	Airbag deployment, pacemaker, flight control
Soft real-time	Performance degradation (multimedia glitches)	Video playback, audio streaming, games

Key RTOS Features

- **Deterministic scheduling** (fixed-priority preemptive, like Rate-Monotonic)
- **Low interrupt latency** (short time from interrupt to handler start)
- **Minimal blocking** (priority inheritance to prevent priority inversion)
- **Predictable memory allocation** (avoid dynamic allocation or provide deterministic allocators)

Examples: VxWorks and FreeRTOS

Feature	VxWorks (Wind River)	FreeRTOS (Amazon)
Type	Commercial, hard real-time	Open source, lightweight
Kernel	Monolithic with modular extensions	Microkernel-like (minimal core)

Typical hardware	PowerPC, ARM, x86 (high-end)	ARM Cortex-M, AVR, small MCUs
Memory footprint	Hundreds of KB to MB	As low as 4-10 KB
Scheduling	Priority preemptive, round-robin, time slicing	Priority preemptive, cooperative option
Use cases	Mars rovers, aerospace, industrial robots	IoT sensors, consumer electronics, hobbyist projects

Real-life analogy:

- **Hard real-time:** A car's airbag – must deploy within milliseconds of impact; even 1 ms late could be fatal.
- **Soft real-time:** A music player – a 50 ms delay in decoding is annoying but not catastrophic.

Embedded Operating Systems

Embedded OSes run on specialised devices with limited resources (CPU, memory, power). They are optimised for **efficiency**, **reliability**, and **real-time** behaviour.

Characteristics

- **Resource constraints:** KB to MB of RAM, MHz processors.
- **Single purpose:** Often runs one application forever (e.g., microwave, thermostat).
- **Static configuration:** No user installation or dynamic loading.
- **Low boot time:** Typically seconds or less.

Common Embedded OSes

OS	Typical use
FreeRTOS	Microcontrollers, IoT sensors
Zephyr	Linux Foundation, wearables, embedded
Embedded Linux	Routers, set-top boxes, infotainment (customisable but heavier)

RT-Linux (PREEMPT_RT)	Linux with real-time patches for industrial control
ThreadX (Azure RTOS)	High-reliability (space, medical devices)

Example: Embedded Linux vs RTOS

- **Embedded Linux** (Yocto, Buildroot): Full OS, networking, drivers, but larger (1-10 MB kernel). Non-deterministic unless patched.
- **FreeRTOS**: Minimal kernel (4-10 KB), deterministic, but fewer features.

Real-life analogy:

- **Embedded Linux**: A mini-PC inside a car dashboard – powerful but takes time to boot.
- **FreeRTOS**: A simple microcontroller in a digital watch – tiny, instant-on, but only does one thing well.

Mobile Operating System Features

Mobile OSes (Android, iOS) extend general-purpose OS features with constraints: **battery life**, **sensor integration**, and **user mobility**.

Power Management

Battery capacity is limited. Mobile OSes aggressively save power:

Technique	Description
CPU throttling	Reduce frequency or core count when idle (DVFS – Dynamic Voltage & Frequency Scaling).
Suspend states	Deep sleep (Doze mode on Android, App Nap on iOS). CPU stops, only wake-on interrupts.
Background restrictions	Limit network and CPU for background apps (Android Background Execution Limits, iOS Background App Refresh).
Battery-aware scheduling	EAS (Energy-Aware Scheduling) on Linux/Android: schedule tasks on efficient cores.

Sensor Integration

Mobile OSes provide frameworks for hardware sensors:

Sensor	Typical use	OS abstraction
Accelerometer	Screen rotation, step counting	SensorManager (Android), CoreMotion (iOS)
Gyroscope	Game orientation, AR	Same
GPS	Location services	LocationManager
Camera	Photos, barcode scanning	Camera API
Fingerprint	Biometric authentication	BiometricPrompt

Other Mobile-Specific Features

- **App sandboxing:** Each app runs as its own user (UID) with restricted permissions (Android UID isolation, iOS App Sandbox).
- **Push notifications:** Wake app without keeping it running (Google FCM, Apple APNs).
- **Seamless updates:** A/B partitions – update while system runs; reboot to new version (Android, iOS).

Real-life analogy: A smartphone is like a Swiss Army knife – many tools in one, but the battery is the limited resource. Power management is like the knife auto-closing tools when not in use to save energy.

System Performance Profiling (perf, DTrace)

Profiling tools help developers and administrators understand where a system spends time and resources.

perf (Linux)

`perf` is the Linux standard profiling tool. It uses hardware performance counters (CPU cycles, cache misses, branches) and software events (context switches, page faults).

Common commands:

```
perf stat ls          # Counts hardware events for a command
perf record -g ls     # Samples stack traces (graph)
perf report           # View results
perf top              # Live view of hottest functions
```

Output example: CPU cycles, instructions per cycle, cache misses, branch mispredictions.

DTrace (Solaris, BSD, macOS, limited Linux)

Created by Sun Microsystems, DTrace allows **dynamic instrumentation** – you can trace kernel or user functions without recompiling or restarting the system. Uses D scripting language.

Use cases:

- System call tracing (`dtrace -n 'syscall:::entry { printf("%s\n", probefunc); }'`)
- File I/O latency analysis
- Memory allocation tracking

Other profiling tools

Tool	Platform	Purpose
strace	Linux	Trace system calls
ltrace	Linux	Trace library calls
Valgrind	Linux	Memory profiling, cache simulation
eBPF/bpftrace	Linux	Modern dynamic tracing (superset of DTrace concepts)
Instruments	macOS/iOS	GUI performance tools (CPU, memory, energy)
Android Studio Profiler	Android	CPU, memory, network profiling

Real-life analogy:

- **perf:** A fuel consumption meter in a car – tells you how much fuel (CPU cycles) each part of your journey uses.

- **DTrace:** A dashboard with meters you can attach to any component on-the-fly without stopping the car (dynamic instrumentation).

Fault Tolerance and Checkpointing

Fault tolerance is the ability of a system to continue operating correctly despite failures (hardware, software, network). **Checkpointing** is a key technique for fault tolerance.

Checkpointing

Periodically save the full state of a process (memory, registers, open files) to stable storage. If the process crashes, it can be **restarted from the last checkpoint** (rollback recovery).

Types:

Type	Description	Overhead
Application-level	Programmer inserts checkpoint calls in code.	Low (targeted)
System-level	OS transparently checkpoints processes (e.g., CRIU in Linux).	High (saves entire memory)
Incremental	Only save pages modified since last checkpoint.	Medium

CRIU (Checkpoint/Restore In Userspace)

Linux tool that can freeze a running container or process, checkpoint its state to disk, and later restore it – possibly on a different machine.

Use cases: Live migration of containers, fast snapshotting for debugging.

Other Fault Tolerance Techniques

Technique	Description
Replication	Run multiple identical copies; vote on outputs.
Primary/backup	One active node; standby takes over on failure (heartbeats).
Transactions (ACID)	Atomic, consistent, isolated, durable operations – rollback on failure.

RAID (disk)	Redundant disks tolerate drive failure (Chapter 9).
Watchdog timers	Reset system if a process becomes unresponsive (common in embedded/RTOS).

Failure Models

Distributed systems classify failures:

- **Fail-stop:** Node halts cleanly; others can detect.
- **Byzantine failure:** Node behaves arbitrarily (malicious, buggy). Requires Byzantine fault tolerance (BFT) – expensive.
- **Network partition:** Subset of nodes cannot communicate.

Real-life analogy: A pilot's flight recorder (black box) – in case of crash, you restart from recorded data? Actually, checkpointing is like saving a video game every few minutes. If the game crashes, you restart from the last save, not from the beginning.

Summary

Topic	Key takeaway
RTOS	Guarantees timing deadlines; VxWorks (hard real-time, commercial), FreeRTOS (lightweight, open source).
Embedded OS	Resource-constrained, single-purpose; FreeRTOS, Embedded Linux.
Mobile OS features	Power management (DVFS, background limits), sensor frameworks, app sandboxing.
System profiling	perf (Linux hardware counters), DTrace (dynamic instrumentation).
Checkpointing	Save process state periodically; CRIU enables container live migration.
Fault tolerance	Replication, primary/backup, transactions, RAID, watchdogs.

Advanced topics demonstrate the breadth of operating systems beyond the desktop. Real-time constraints, power budgets, performance tuning, and fault resilience are critical in modern computing – from tiny sensors to cloud data centres.